UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Jhuan Luis Almeida Assumpção

ANÁLISE COMPARATIVA DE MODELOS TEXT-TO-SQL PARA CONSULTAS EM BANCOS DE DADOS EDUCACIONAIS BRASILEIROS DO ENSINO SUPERIOR

Santa Maria, RS
2025

Jhuan Luis Almeida Assumpção

ANÁLISE COMPARATIVA DE MODELOS TEXT-TO-SQL PARA CONSULTAS EM BANCOS DE DADOS EDUCACIONAIS BRASILEIROS DO ENSINO SUPERIOR

Trabalho Final de Graduação apresentado ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Santa Maria (UFSM, RS), como requisito parcial para obtenção do grau de **Bacharel em Ciência da Computação**.

ORIENTADORA: Profa. Andrea Schwertner Charão

Santa Maria, RS
2025

Jhuan Luis Almeida Assumpção

ANÁLISE COMPARATIVA DE MODELOS TEXT-TO-SQL PARA CONSULTAS EM BANCOS DE DADOS EDUCACIONAIS BRASILEIROS DO ENSINO SUPERIOR

Trabalho Final de Graduação apresentado ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Santa Maria (UFSM, RS), como requisito parcial para obtenção do grau de **Bacharel em Ciência da Computação**.

Aprovado em 07 de Julho de 2025:

Andrea Schwertner Charão (UFSM)
(Presidente/Orientadora)

Marcelo Lopes Kroth (UFSM)

Sergio Luis Sardi Mergen (UFSM)

Santa Maria, RS
2025

AGRADECIMENTOS

Quero dedicar esse trabalho às mulheres incríveis da minha vida: minha vó Dona Zuzu, minha mãe Adilma Almeida, minha irmã Emily Massariol e minha namorada Iolanda Henz. Mesmo de longe, sei que minha vó está sempre orando e torcendo por mim, e isso me dá ainda mais força pra seguir em frente. Obrigado a cada uma de vocês por todo o amor, apoio e por sempre acreditarem em mim. Essa conquista também é de vocês. E estendo essa gratidão a toda a minha família, que sei que sempre torceu por mim e desejou o meu sucesso.

Agradeço também aos meus amigos e colegas de graduação Luana, Gilson e Roberto. A companhia de vocês fez toda a diferença nessa caminhada, pelas conversas, pelas risadas e pelos momentos que tornaram tudo mais leve e especial. Vou guardar com muito carinho tudo o que vivemos juntos durante esses anos.

E um agradecimento especial à minha orientadora, por todo o apoio, paciência e dedicação na construção deste trabalho.

ANÁLISE COMPARATIVA DE MODELOS TEXT-TO-SQL PARA CONSULTAS EM BANCOS DE DADOS EDUCACIONAIS BRASILEIROS DO ENSINO SUPERIOR

COMPARATIVE ANALYSIS OF TEXT-TO-SQL MODELS FOR QUERIES IN BRAZILIAN HIGHER EDUCATION DATABASES

Jhuan Luis Almeida Assumpção[1], **Andrea Schwertner Charão**[2]

RESUMO

O acesso a dados armazenados em bancos relacionais, como os do Censo da Educação Superior, frequentemente requer conhecimento técnico em SQL, o que representa uma barreira para usuários não especializados, como gestores educacionais e pesquisadores, limitando a utilização eficiente de informações públicas. A transformação de perguntas em linguagem natural em comandos SQL, conhecida como Text-to-SQL, tem se destacado como uma abordagem promissora para facilitar o acesso a dados, especialmente por usuários sem conhecimento técnico em bancos de dados. Com os avanços dos modelos de linguagem de grande escala (LLMs), surgem novas possibilidades para automatizar essa tarefa de forma precisa e eficiente. Este trabalho avalia a eficácia de LLMs na geração de comandos SQL a partir de perguntas em linguagem natural, utilizando os dados do Censo da Educação Superior. Foram selecionados quatro modelos de código aberto: OneSQL-v0.1-Qwen-7B (Alibaba), sqlcoder-7b-2 (Defog), OmniSQL-7B (ByteDance) e XiYan-SQL (Alibaba Cloud). Os modelos foram testados com perguntas manualmente elaboradas, de variada complexidade, e avaliados quanto à correção sintática e semântica das *queries*, aderência ao esquema relacional, capacidade de generalização e eficiência computacional. Os resultados indicam que todos os modelos realizam a tarefa com satisfatória acurácia, destacando seu potencial para democratizar o acesso a dados públicos educacionais por meio de interfaces baseadas em linguagem natural. Entre eles, o modelo XiYan-SQL apresentou o melhor desempenho geral, tanto em precisão quanto em eficiência. Este estudo contribui para o avanço no uso de LLMs em tarefas de Text-to-SQL e para a promoção da transparência e acessibilidade na gestão educacional.

Palavras-chave: Modelos de Linguagem de Grande Escala, Text-to-SQL, Censo da Educação Superior, Geração de Consultas SQL, Análise de Dados Educacionais

[1] Autor: Graduando em Bacharelado em Ciência da Computação pela Universidade Federal de Santa Maria.

[2] Orientadora: Professora do Departamento de Linguagens e Sistemas de Computação da Universidade Federal de Santa Maria.

ABSTRACT

Accessing data stored in relational databases, such as those from Brazil's Higher Education Census (INEP), often requires technical expertise in SQL, posing a significant barrier for non-specialized users, such as educational managers and researchers, and limiting the effective use of public data. Transforming natural language questions into SQL commands, known as Text-to-SQL, has emerged as a promising approach to facilitate data access, especially for users without technical knowledge of databases. With the recent advancements in large language models (LLMs), new opportunities have arisen to automate this task with greater accuracy and efficiency. This study evaluates the effectiveness of LLMs in generating SQL commands from natural language questions, using data from Brazil's Higher Education Census (INEP). Four open-source models were selected for analysis: OneSQL-v0.1-Qwen-7B (Alibaba), sqlcoder-7b-2 (Defog), OmniSQL-7B (ByteDance), and XiYan-SQL (Alibaba Cloud). The models were tested using manually crafted questions of varying complexity and assessed based on syntactic and semantic correctness of the *queries*, adherence to the relational schema, generalization capability, and computational efficiency. The results show that all models perform the task with satisfactory accuracy, highlighting their potential to democratize access to public educational data through natural language interfaces. Among them, XiYan-SQL achieved the best overall performance, standing out in both accuracy and efficiency. This research contributes to the advancement of LLM applications in Text-to-SQL tasks and promotes transparency and accessibility in educational data management.

Keywords: Large Language Models, Text-to-SQL, Higher Education Census, SQL Query Generation, Educational Data Analysis

LISTA DE FIGURAS

Figura 1 – Estágios da análise no PLN 13
Figura 2 – Codificação da sequência de entrada com representações vetoriais . . . 18
Figura 3 – Árvore sintática de uma consulta SQL 18
Figura 4 – Exemplo de mapeamento semântico entre pergunta e esquema (schema linking) 19
Figura 5 – Geração estruturada de uma consulta SQL organizada em forma de árvore 19
Figura 6 – Os dados do censo são disponibilizados via CSV, com filtragem por ano 21
Figura 7 – O Painel disponibiliza uma interface mais amigável para consulta dos dados do Censo 21
Figura 8 – Fluxo da avaliação das métricas. 35
Figura 9 – Arquitetura do sistema de consulta educacional baseado em linguagem natural com XiYan-SQL. 45

LISTA DE TABELAS

Tabela 1 – Questões de Baixa Complexidade 33
Tabela 2 – Questões de Média Complexidade (Parte 1) 33
Tabela 3 – Questões de Média Complexidade (Parte 2) 34
Tabela 4 – Questões de Alta Complexidade 34
Tabela 5 – Resultados das Consultas de Nível Fácil 40
Tabela 6 – Resultados das Consultas de Nível Médio 41
Tabela 7 – Resultados das Consultas de Nível Difícil 41
Tabela 8 – Resumo das Métricas por Nível de Dificuldade 43

SUMÁRIO

1 Introdução . . . 11
2 Fundamentação e trabalhos relacionados . . . 12
2.1 Processamento de Linguagem Natural . . . 12
2.1.1 Tokenização . . . 13
2.1.2 Análise Léxica . . . 14
2.1.3 Análise Sintática . . . 14
2.1.4 Análise Semântica . . . 14
2.1.5 Análise Pragmática . . . 15
2.2 Modelos de Linguagem de Grande Escala (LLM) . . . 15
2.2.1 *Few-Shot* . . . 16
2.2.2 *Fine-Tuning* . . . 16
2.3 Text-to-SQL (T2S) . . . 16
2.3.1 Fundamentos Text-to-SQL . . . 17
2.3.2 Funcionamento Text-to-SQL . . . 17
2.4 Censo da Educação Superior . . . 20
2.4.1 Dados Coletados . . . 20
2.4.2 Disponibilização dos Dados . . . 20
2.5 Métricas de Avaliação do Text-to-SQL . . . 21
2.5.1 Precisão de Correspondência Exata (EM) . . . 22
2.5.2 Precisão de Execução (EX) . . . 22
2.5.3 Pontuação de Eficiência Válida (VES) . . . 23
2.6 Trabalhos relacionados . . . 23
3 Desenvolvimento . . . 25
3.1 Tecnologias Usadas . . . 25
3.2 Uso do Google Colab com Ambiente T4 . . . 25
3.3 Modelagem do Esquema Relacional . . . 26
3.4 Elaboração das Consultas de Referência (Gabarito) . . . 27
3.5 Conexão e Execução via API Remota . . . 28
3.6 Análise Estrutural da Consulta SQL . . . 30
3.7 Padronização dos Resultados . . . 31
3.8 Análise de Eficiência por Tempo de Execução . . . 31
3.9 Conjunto de Perguntas para Avaliação . . . 32
3.9.1 Integração dos Modelos com o Pipeline . . . 33
3.9.2 Funcionamento do Pipeline em Casos Práticos . . . 35
4 Resultados . . . 37
4.1 Métricas . . . 37
4.1.1 Precisão de Correspondência Exata (EM) . . . 37
4.1.1.1 Detalhamento . . . 37
4.1.1.2 Exemplo Prático . . . 38
4.1.2 Precisão de Execução (EX) . . . 38
4.1.2.1 Detalhamento . . . 38
4.1.2.2 Exemplo Prático . . . 39
4.1.3 Pontuação de Eficiência Válida (VES) . . . 39

4.1.3.1 Detalhamento 39
4.1.3.2 Exemplo Prático 40
4.2 Avaliação de perguntas 40
4.2.1 Nível Fácil 40
4.2.2 Nível Médio 41
4.2.3 Nível Difícil 41
4.3 Síntese Geral 42
5 Conclusão 42
5.1 Arquitetura Proposta para o Sistema de Consulta Educacional . . 44

Referências **46**
APÊNDICE A Esquema do Banco de Dados **49**
APÊNDICE B Perguntas e Consultas SQL de Referência **53**
1 Nível Fácil 53
2 Nível Médio 53
3 Nível Difícil 55

1 INTRODUÇÃO

A quantidade de dados gerados e armazenados em bases estruturadas tem crescido exponencialmente nas últimas décadas, impulsionando o desenvolvimento de tecnologias que facilitem o acesso, a consulta e a análise dessas informações (ANTONI et al., 2021). No contexto educacional brasileiro, o Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP) disponibiliza anualmente grandes volumes de dados públicos, como os provenientes do Censo da Educação Superior, que representam uma fonte rica de informações sobre instituições, cursos, docentes e discentes. A exploração desses dados por parte de pesquisadores, gestores públicos e cidadãos interessados ocorre de diversas formas, como *dashboards*, relatórios e arquivos CSVs, que oferecem uma ampla variedade de abordagens de análise. Apesar disso, a consulta em linguagem natural, que permitiria acessar informações de forma intuitiva sem a necessidade de conhecimentos técnicos em linguagens de consulta como *SQL (Structured Query Language)* (GUPTA et al., 2017), permanece uma abordagem não consolidada no contexto brasileiro.

Diante dessa lacuna, os recentes avanços em modelos de linguagem de grande escala *(Large Language Models - LLMs)*, como os baseados em arquiteturas *Transformer*, têm aberto novas possibilidades para tornar esse tipo de informação mais acessível (VASWANI et al., 2023). LLMs treinados em grandes volumes de dados textuais têm demonstrado capacidade de interpretar, gerar e traduzir linguagem humana com altos níveis de coerência e precisão. Um campo específico que tem se beneficiado desses avanços é o de Text-to-SQL, que consiste na tradução de comandos em linguagem natural para instruções SQL capazes de recuperar dados relacionais (SHI et al., 2024).

Apesar desses avanços, a aplicação de LLMs para consulta de dados educacionais públicos em português, especialmente no contexto brasileiro, ainda é pouco explorada. Um estudo de 2021, conduzido por pesquisadores da Universidade de São Paulo, apresenta o mRAT-SQL+GAP, um sistema de Text-to-SQL adaptado para português, utilizando uma versão traduzida do *dataset Spider*. Esse trabalho, apoiado por IBM e FAPESP, testou modelos como mBART-50 e BERTimbau, alcançando resultados promissores, como 59,5% de acurácia em testes em português (JOSÉ; COZMAN, 2021). Entretanto, ainda são escassas as investigações direcionadas a bases específicas como o Censo da Educação Superior, o que evidencia uma oportunidade relevante para pesquisas que explorem como modelos existentes podem ser aplicados na democratização do acesso a informações educacionais.

Este trabalho propõe uma investigação exploratória sobre a aplicabilidade de LLMs na geração automática de comandos SQL a partir de perguntas formuladas em linguagem natural, utilizando como base os dados estruturados do Censo da Educação Superior. A pesquisa foca na avaliação comparativa do desempenho quatro modelos de código aberto voltados à tarefa de Text-to-SQL: OneSQL-v0.1-Qwen-7B, da Alibaba; sqlcoder-7b-2, da Defog; OmniSQL-7B, da Renmin University of China + ByteDance; e XiYan-SQL, desenvolvido pela Alibaba Cloud com foco específico em recuperação de dados via SQL. Esses modelos foram selecionados por representarem diferentes arquiteturas, tamanhos e estratégias de treinamento, oferecendo um panorama abrangente sobre o estado da arte atual em modelos voltados a essa tarefa. Além disso, são modelos de código aberto, o que os torna passíveis de uso em ambientes mais acessíveis, sem depender de soluções proprietárias ou com custos elevados.

A proposta metodológica do estudo concentra-se unicamente na avaliação técnica

da geração de SQL a partir de perguntas em linguagem natural, não incluindo o desenvolvimento de uma interface interativa. As perguntas foram construídas manualmente com base em esquemas relacionais derivados do Censo da Educação Superior e classificadas por níveis de complexidade. A avaliação dos resultados contempla aspectos como correção sintática e semântica das consultas geradas, aderência ao esquema de dados, capacidade de generalização e desempenho computacional. Essa abordagem segue práticas comuns na literatura, como as descritas em pesquisas recentes, que utilizam métricas como *Exact Matching Accuracy (EM)*, *Execution Accuracy (EX)* e *Valid Efficiency Score (VES)* para avaliar modelos Text-to-SQL (ZHU et al., 2024).

Com essa abordagem, busca-se avaliar a capacidade desses modelos em lidar com a complexidade da linguagem natural em português e utilizando os dados educacionais brasileiros. Os resultados esperados incluem a identificação de modelos eficazes para essa tarefa, bem como a compreensão de suas limitações e possíveis melhorias, contribuindo assim para a democratização do acesso aos dados públicos e para a transparência na gestão educacional.

A motivação central deste trabalho reside na possibilidade de contribuir para a construção de soluções tecnológicas mais acessíveis para o público em geral, permitindo que bases públicas de dados, como as educacionais, sejam consultadas de forma mais intuitiva, sem a exigência de conhecimento técnico avançado. Além disso, os resultados da pesquisa podem servir de base para futuros trabalhos que envolvam a integração desses modelos em aplicações práticas voltadas à gestão educacional, transparência pública ou pesquisas científicas.

O restante deste trabalho está organizado como segue. Na Seção 2, são apresentados os fundamentos teóricos e os trabalhos relacionados, abordando desde conceitos de Processamento de Linguagem Natural até as métricas utilizadas na avaliação de modelos Text-to-SQL. Na Seção 3, é descrito o processo de desenvolvimento, incluindo as tecnologias utilizadas, a preparação do ambiente e a execução dos testes. Na Seção 4, é realizada a avaliação final, com análise das métricas aplicadas e discussão dos resultados observados. Na Seção 5, são apresentadas as conclusões obtidas a partir dos resultados alcançados, bem como uma reflexão sobre as limitações e contribuições do estudo. Além disso, é proposta uma arquitetura que poderá servir como base para trabalhos futuros. Por fim, são disponibilizados as referências utilizadas.

2 FUNDAMENTAÇÃO E TRABALHOS RELACIONADOS

Para estabelecer o contexto desta pesquisa, esta seção apresenta uma revisão das tecnologias empregadas, da fonte de dados de interesse e dos trabalhos correlatos que fundamentam e dialogam com o estudo proposto.

2.1 Processamento de Linguagem Natural

O Processamento de Linguagem Natural (PLN) é um campo de estudo que explora maneiras de ensinar computadores a processar e interagir com a linguagem humana, incluindo textos e fala, que integra conhecimentos da linguística, da ciência da computação e da inteligência artificial, com o objetivo de desenvolver métodos e algoritmos capazes de analisar, interpretar e gerar a linguagem humana, seja ela apresentada na forma de texto ou de fala (JURAFSKY; MARTIN, 2008). Essa interdisciplinaridade permite que o PLN

aborde a comunicação em seus diversos níveis, explorando desde aspectos morfológicos e sintáticos até questões semânticas e pragmáticas.

Conforme destacado por Liddy (2001), enquanto linguagens de programação são projetadas de maneira a evitar ambiguidades através de regras rigorosas, a linguagem natural exige abordagens que consigam capturar a riqueza e a variabilidade presentes na comunicação humana. Nesse sentido, as técnicas aplicadas ao PLN combinam abordagens estatísticas e algoritmos de aprendizado de máquina com modelos linguísticos. Segundo Dale (2010) o processamento de um texto em linguagem natural geralmente é dividido em alguns estágios, como mostra a Figura 1.

Figura 1 – Estágios da análise no PLN

Fonte: Adaptado de Dale (2010)

2.1.1 Tokenização

A tokenização é o processo de segmentar um texto em unidades menores, como palavras ou símbolos. Em linguagens naturais, sem regras fixas, o processo não ocorre da mesma forma que em linguagens artificiais, como as de programação. Em línguas delimitadas por espaço, como o português, espaços sugerem limites de palavras, mas pontuação, como em "Dr. Silva.", pode indicar abreviações ou fim de sentença, exigindo tokenizadores precisos (WEBSTER; KIT, 1992).

Em linguagens não segmentadas, como o chinês, a ausência de delimitadores torna a tokenização dependente de informações léxicas (SPROAT; SHIH, 1990). A tokenização prepara o texto para a análise léxica, que explora o significado de cada *token*, abrindo caminho para análises de estrutura e sentido.

2.1.2 Análise Léxica

A análise léxica, etapa posterior à tokenização, conecta palavras do texto aos seus *lemmas*, formas canônicas presentes em dicionários (HIPPISLEY, 2010). Um *lemma* é o termo abstrato que engloba variantes morfológicas, como o substantivo "livro" para formas como "livros" ou "livrinho". Esse processo depende de dicionários de *lemmas* com informações semânticas e sintáticas para realizar esse mapeamento.

A análise léxica envolve dois processos: o *parsing*, que associa uma palavra ao seu *lemma*, e a geração morfológica, que deriva palavras a partir do *lemma* (HIPPISLEY, 2010). Em recuperação de informação, o *parsing* suporta o *stemming*, reduzindo palavras ao seu radical *(stem)*, como "livr" para "livro". Isso agrupa variantes morfológicas, simplificando a extração de termos-chave. Assim, a análise léxica pavimenta o caminho para a análise sintática, que investiga relações gramaticais entre palavras (MANNING; SCHUTZE, 1999).

2.1.3 Análise Sintática

A análise sintática identifica a estrutura gramatical das frases, essencial para extrair seu significado, que expressa proposições ou ideias sobre o mundo (CHOMSKY, 2014). Diferentemente de uma sequência linear de palavras, a frase exige uma representação hierárquica, comumente modelada por gramáticas e árvores sintáticas (SANDRA, 1994).

Gramáticas definem regras que geram árvores sintáticas, que organizam os elementos de uma frase em uma hierarquia. Essa hierarquia é geralmente representada por árvores sintáticas, nas quais cada nó representa uma categoria gramatical (como sintagma nominal ou verbal) e os ramos indicam como esses constituintes se combinam. A construção da árvore parte de um nó raiz, que representa a sentença completa, e se ramifica até os termos mais básicos, como pronomes, verbos e substantivos. Esse processo revela como as palavras estão relacionadas estruturalmente, o que é essencial para a etapa seguinte da análise semântica, que interpreta o significado global da frase (SCHÜTZE; MANNING; RAGHAVAN, 2008).

2.1.4 Análise Semântica

A análise semântica em Processamento de Linguagem Natural (PLN) interpreta o significado de palavras, expressões, sentenças e enunciados no contexto, sendo crucial para aplicações como recuperação de informação, extração de dados, sumarização, mineração de textos e tradução automática (GODDARD; SCHALLEY, 2010). Diferentemente da análise sintática, que foca na estrutura gramatical, a semântica traduz essas estruturas em representações de significado, frequentemente usando metalinguagens.

O processo depende das análises léxicas e sintáticas prévias, integrando informações contextuais, conhecimento lexical e raciocínio para "entender" o enunciado, seja incorporando-o a uma base de conhecimento ou desencadeando ações (CRUSE, 2004). Na linguística, a semântica examina interpretações de falantes nativos, padrões de uso e implicações, apoiando-se em técnicas de análise de textos para identificar colocação e frequência.

2.1.5 Análise Pragmática

A análise pragmática vai além da estrutura de uma única frase, buscando nas frases adjacentes e no contexto a compreensão de elementos como referências pronomiais, coerência textural e intenção discursiva (MÜLLER, 2003). Diferentemente da análise semântica, que foca no significado literal, a pragmática interpreta o uso da linguagem em situações específicas, considerando intenções e implicações.

O processo depende de análises léxicas, sintática e semântica prévias, integrando pistas contextuais e conhecimento discursivo para resolver ambiguidades, como referências pronominais (e.g., em "Ana deixou o livro na mesa. Ao vê-lo, Pedro o pegou", determinar a quem "lo" e "o" se referem) (CRUSE, 2004). Estruturas comuns na análise pragmática incluem gramáticas baseadas em casos que identificam padrões semânticos não terminais para reconhecer construções frasais em contextos específicos (MÜLLER, 2003).

2.2 Modelos de Linguagem de Grande Escala (LLM)

Modelos de Linguagem de Grande Escala (LLMs) representam um avanço transformador em inteligência artificial, impulsionados pela escalabilidade de arquiteturas baseadas em *Transformers* e pela disponibilidade de grandes volumes de dados textuais. Esses modelos, como GPT, LLaMA e DeepSeek, são pré-treinados em vastos conjuntos de textos, adquirindo capacidades emergentes, como aprendizado *few-shot* e *instruction following*, que os tornam altamente adaptáveis em diversas tarefas (WEI et al., 2022a).

LLMs operam com base na arquitetura *Transformer*, introduzida por Vaswani et al. (2017), que revolucionou o processamento de sequências de dados ao substituir redes recorrentes por mecanismos de atenção. A arquitetura *Transformer* é composta por duas partes principais: um codificador, que mapeia a entrada para uma representação contextual, e um decodificador, que gera a saída a partir dessa representação. O núcleo da arquitetura é o mecanismo da atenção, especificamente a *scaled dot-product attention*, que calcula a relevância de cada *token* em uma sequência em relação aos demais, permitindo capturar dependências contextuais de longo alcance (VASWANI et al., 2017). Em LLMs, como os modelos GPT, utiliza-se predominantemente a parte do decodificador (*decoder-only*), otimizada para tarefas de geração autorregressiva, enquanto modelos como BERT exploram o codificador para tarefas de compreensão (DEVLIN et al., 2019).

Durante o pré-treinamento, os LLMs são expostos a grandes volumes de dados textuais, ajustando seus parâmetros para prever a próxima palavra da sequência, um processo conhecido como modelagem autorregressiva (SHI et al., 2024). Essa abordagem permite que LLMs aprendam padrões linguísticos complexos, desde gramática até semântica contextual. Na inferência, os modelos geram texto *token* por *token*, selecionando a palavra com maior probabilidade condicional com base no contexto fornecido pelo *prompt* (WEI et al., 2022a). Em tarefas como Text-to-SQL, o *prompt* inclui esquemas de banco de dados e perguntas em linguagem natural para que o LLM mapeie essas entradas para consultas SQL estruturadas (SHI et al., 2024), como por exemplo:

```
SELECT city_name FROM cities WHERE population > 1000000
```

Diferentemente de modelos tradicionais, como *Seq2Seq*, que exigem pipelines específicas para tarefas como tradução ou geração de texto, LLMs operam em uma arquitetura unificada que permite generalização para aplicações variadas (VASWANI et al., 2023). Em diversas áreas, LLMs são amplamente utilizados em tarefas como geração

de texto, resposta a perguntas, sumarização, tradução automática e até mesmo tarefas estruturadas, como Text-to-SQL, onde convertem linguagem natural em consultas SQL.

2.2.1 *Few-Shot*

O aprendizado *few-shot* refere-se à capacidade dos LLMs de realizar tarefas fornecendo apenas alguns exemplos no *prompt*, sem a necessidade de grandes *datasets* de treinamento, como exigido por modelos tradicionais (BROWN et al., 2020). Por exemplo, em Text-to-SQL, um LLM pode gerar consultas corretas após receber dois ou três pares de perguntas e suas respectivas consultas SQL no *prompt*, adaptando-se rapidamente ao contexto (WEI et al., 2022a). Além disso, o seguimento de instruções (*instruction following*) possibilita respostas a comandos complexos sem treinamento adicional.

A aplicação de LLMs é otimizada por duas abordagens principais: *prompt engineering* e *fine-tuning*. O *prompt engineering* explora a capacidade de seguimento de instruções dos LLMs, estruturando *prompts* para maximizar a quantidade de respostas. Técnicas como *Chain-of-Thought (CoT)*, que incentiva o raciocínio passo a passo, e *Retrieval-Augmented Generation (RAG)*, que incorpora conhecimento externo, melhoram o desempenho em tarefas complexas (WEI et al., 2022b). Em Text-to-SQL, por exemplo, *prompts* bem estruturados, que incluem esquemas de banco de dados e exemplos, aumentam a precisão da geração de consultas (SHI et al., 2024).

2.2.2 *Fine-Tuning*

O *fine-tuning*, por outro lado, adapta LLMs pré-treinados a domínios específicos ,ajustando parâmetros em *datasets* especializados. Métodos como *LoRA (Low-Rank Adaptation)* permitem *fine-tuning* eficiente, reduzindo custos computacionais (HU et al., 2022). O *fine-tuning* é comum em tarefas que exigem alta precisão, como classificação de sentimentos ou extração de entidades nomeadas, mas requer *datasets* robustos para evitar *overffiting* (HOWARD; RUDER, 2018).

Em Text-to-SQL, espera-se que LLMs sejam integrados a sistemas de banco de dados em tempo real, permitindo a geração de consultas SQL mais precisas e adaptáveis a esquemas dinâmicos, como os encontrados em plataformas como *BigQuery* ou *PostgreSQL* (QIN et al., 2022). A criação de *datasets* mais representativos, como *Spider 2.0*, também impulsionará o desenvolvimento de modelos capazes de lidar com cenários empresariais complexos (YU et al., 2018).

2.3 Text-to-SQL (T2S)

O acesso simplificado a dados estruturados em bancos de dados relacionais é uma demanda crescente em diversos contextos, onde usuários buscam extrair informações sem a necessidade de dominar linguagens técnicas de consulta. Nesse cenário, o Text-to-SQL, uma tarefa de inteligência artificial, destaca-se por converter perguntas formuladas em linguagem natural em consultas *Structured Query Language (SQL)*, permitindo que usuários não especialistas interajam com o banco de dados de forma eficiente (QIN et al., 2022). Por exemplo, em um contexto educacional, alguém pode perguntar “Quais instituições oferecem cursos de Tecnologia em 2025” e obter a consulta SQL:

```
SELECT Nome FROM Instituicoes WHERE EXISTS (SELECT * FROM Cursos
    WHERE InstituicaoID = Instituicoes.InstituicaoID AND Tipo = '
    Tecnologia' AND Ano = 2025)
```

sem precisar compreender a sintaxe da linguagem. Essa tarefa, que envolve tradução semântica e geração estruturada, é essencial para facilitar o acesso a dados em esquemas complexos, como os do Censo da Educação Superior, que abrangem informações sobre matrículas, cursos e instituições.

2.3.1 Fundamentos Text-to-SQL

O funcionamento do Text-to-SQL (T2S) pode ser descrito como um problema de mapeamento de sequência-para-sequência, no qual uma pergunta em linguagem natural é representada como uma lista de *tokens* e um esquema de banco de dados é composto por um conjunto de tabelas e colunas. Esses elementos são então transformados em uma consulta SQL estruturada.

Em cenários de turno único, cada pergunta é traduzida de forma isolada, sem levar em conta interações anteriores. O objetivo é gerar a consulta SQL que melhor corresponde à pergunta e ao esquema, utilizando técnicas como *schema linking* para alinhar os *tokens* da pergunta aos componentes do esquema e arquiteturas *Transformer* para capturar o contexto semântico de maneira pontual.

Já em cenários de multi-turno, modela-se uma sequência de interações entre pergunta e consulta, de modo que cada novo par pergunta-consulta dependa do histórico das interações anteriores e do mesmo esquema de banco de dados. Nesse caso, o sistema precisa lidar com desafios adicionais, tais como:

- **Resolução de elipses e anáforas**: perguntas subsequentes podem referir-se a entidades ou filtros aplicados em turnos anteriores.
- ***History modeling***: incorporar o contexto passado para garantir coerência e consistência na geração de cada consulta.
- ***Schema linking* incremental**: alinhar novos *tokens* tanto ao esquema original quanto aos resultados ou condições introduzidos nos turnos prévios.

2.3.2 Funcionamento Text-to-SQL

O processo de Text-to-SQL ocorre em quatro etapas principais: tokenização da entrada, codificação contextual, mapeamento semântico (*schema linking*) e geração estruturada da consulta. Essas etapas têm como função orientar a geração de consultas sintaticamente válidas e semanticamente alinhadas. Na etapa de tokenização, a pergunta e o esquema são concatenados em uma sequência unificada, utilizando *tokens* especiais, como `[CLS]`, que indica o início da sequência, e `[SEP]`, que delimita diferentes segmentos da entrada. Por exemplo, ao utilizar a pergunta "Quais cursos em 2023?" e o esquema (S): `Curso (CursoId, Nome, Ano)`, a sequência tokenizada fica da seguinte forma: `[CLS] Quais cursos em 2023 ? [SEP] Cursos CursoID Nome Ano [SEP]`. Essa sequência é então processada pelo codificador, que gera representações vetoriais iniciais para cada *token*, capturando sua semântica (QIN et al., 2022) (veja a Figura 2).

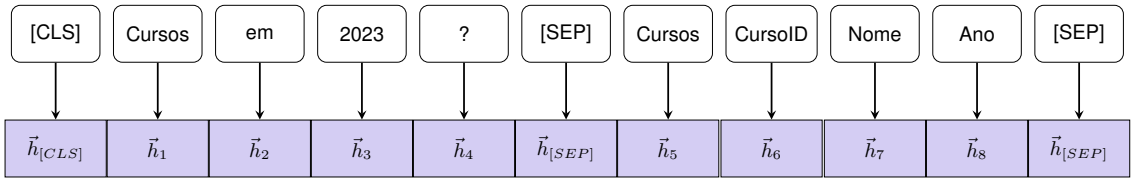

Figura 2 – Codificação da sequência de entrada com representações vetoriais

Fonte: Adaptado de Devlin et al. (2019)

A codificação contextual utiliza o mecanismo de atenção para criar representações que capturam as relações entre os *tokens* da pergunta e do esquema. O modelo calcula escores de atenção, destacando conexões como "instituições" com a tabela Instituicoes e "cursos de Tecnologia" com a tabela Cursos. Métodos como RAT-SQL introduzem *relation-aware self-attention*, considerando informações estruturais, como a relação entre InstituicaoID em Instituicoes e Cursos, para melhorar o alinhamento semântico (WANG et al., 2019). Essa etapa é essencial para modelar a estrutura hierárquica do SQL, que segue uma gramática formal representada por árvores sintáticas (veja a Figura 3), alinhando-se à sua preferência por representações hierárquicas mencionada anteriormente (QIN et al., 2022). A codificação permite que o modelo compreenda o contexto global da entrada, preparando-a para o mapeamento semântico.

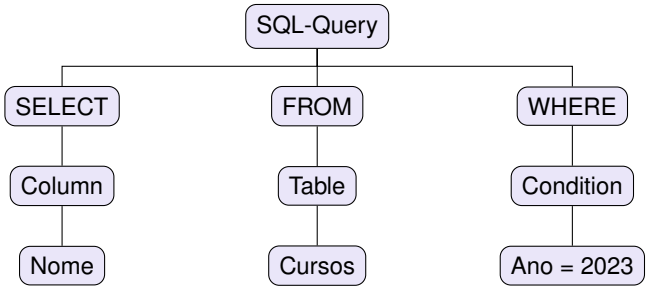

Figura 3 – Árvore sintática de uma consulta SQL

Fonte: Autor

O mapeamento semântico, ou *schema linking*, combina as palavras da pergunta com as tabelas e colunas corretas no esquema. É como um tradutor associando termos em uma língua a conceitos em outra. Na pergunta "Quais instituições oferecem cursos de Tecnologia em 2023?", o sistema identifica que "instituições" se refere à tabela Instituicoes, "cursos de Tecnologia" à tabela Cursos com a condição Tipo = 'Tecnologia', e "2023" à coluna Ano (veja a Figura 4). Métodos como GRAPPA ajudam a fazer essas conexões com precisão, mesmo quando as tabelas são complexas (YU et al., 2020), como em bases educacionais onde Instituicoes e Cursos estão interligadas. Esse passo é crucial para evitar erros em esquemas detalhados.

A geração estruturada da consulta é o momento em que o sistema cria a consulta SQL, palavra por palavra, como montar uma frase correta em um idioma específico. O sistema segue a gramática do SQL, garantindo que a consulta seja válida, como seguir uma receita para evitar erros. Por exemplo, ele decide quais palavras, como *SELECT* ou *WHERE*, devem ser usadas e em que ordem. Essa geração é organizada como uma árvore, onde cada parte da consulta (como condições ou junções) se conecta de forma hierárquica,

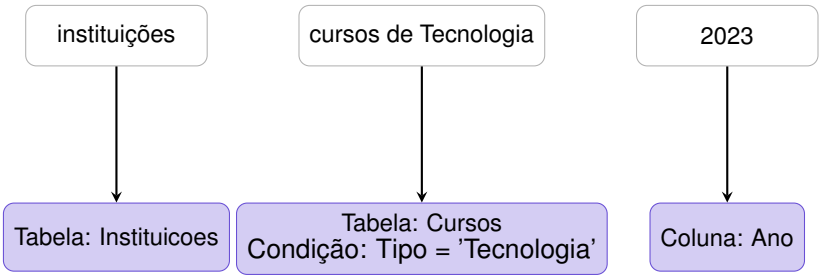

Figura 4 – Exemplo de mapeamento semântico entre pergunta e esquema (schema linking)

Fonte: Autor

refletindo a estrutura do SQL (YU et al., 2018) (veja a Figura 5). O sistema aprende a criar consultas corretas com base em exemplos, ajustando-se para produzir resultados precisos (QIN et al., 2022).

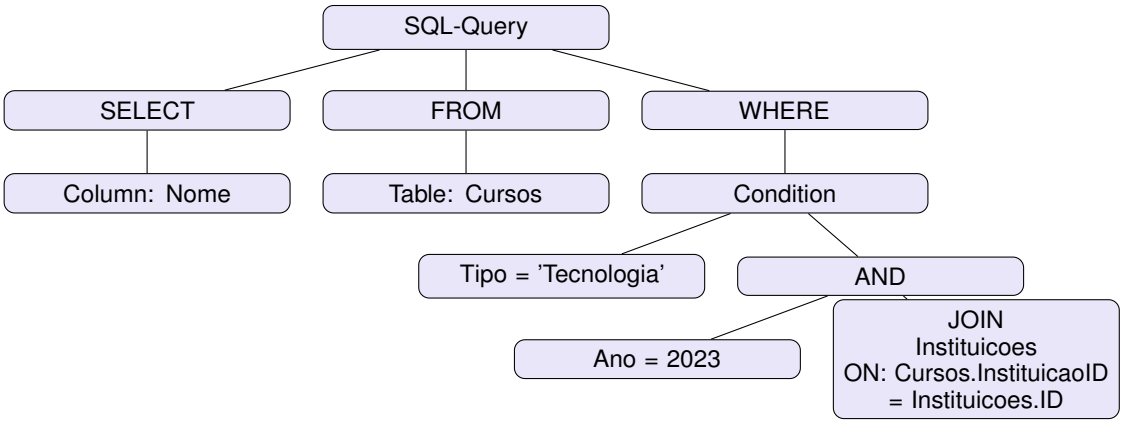

Figura 5 – Geração estruturada de uma consulta SQL organizada em forma de árvore

Fonte: Autor

Datasets como *Spider 2.0* (YU et al., 2018) e *BIRD* (LI et al., 2023) têm ampliado significativamente a diversidade e representatividade dos esquemas de banco de dados para tarefas de *Text-to-SQL*. O *Spider 2.0*, uma evolução do *Spider 1.0*, introduz 632 problemas complexos baseados em cenários empresariais reais, abrangendo bases de dados extensas com mais de 1.000 colunas, armazenadas em sistemas como BigQuery, Snowflake e SQLite. Ele inclui esquemas com múltiplas tabelas, consultas aninhadas e condições complexas, refletindo casos de uso educacionais e industriais, como análises de dados financeiros ou acadêmicos. Essa diversidade desafia os modelos a lidar com estruturas relacionais intricadas e esquemas heterogêneos, promovendo a generalização em diferentes domínios. Por outro lado, o *BIRD* foca na robustez e na complexidade de consultas em cenários realistas, oferecendo 12.751 pares de perguntas e consultas SQL em 95 bases de dados de grande escala, muitas delas extraídas de contextos práticos, como gestão educacional e censo populacional. Ele incorpora desafios adicionais, como ruído nos dados e ambiguidades nas perguntas, tornando-o ideal para avaliar a capacidade dos modelos em cenários próximos ao mundo real. Métodos como MAGIC (ASKARI; POELITZ; TANG, 2025) e SuperSQL utilizam estratégias de autocorreção e agentes colaborativos para melhorar a precisão, o que é particularmente útil para consultas em bases complexas,

como as do Censo. O suporte a cenários *cross-linguísticos*, como o *dataset* DuSQL, facilita a aplicação do *Text-to-SQL* em contextos globais, permitindo a geração de consultas a partir de perguntas em diferentes idiomas.

A avaliação do desempenho desses sistemas é realizada por métricas específicas, como *Exact Matching Accuracy, (EM)*, que verifica a equivalência sintática entre a consulta gerada e a referência, e *Execution Accuracy (EA)*, que avalia a correção dos resultados retornados pelo banco de dados e *Valid Efficiency Score (VES)*, que avalia a eficiência de consultas válidas em termos de desempenho e complexidade. Essas métricas são amplamente adotadas em *datasets* como *Spider*.

2.4 Censo da Educação Superior

O Censo da Educação Superior, realizado anualmente pelo Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), órgão vinculado ao Ministério da Educação (MEC) do Brasil, é um levantamento que sistematiza dados estatísticos e qualitativos sobre o ensino superior no país. Seu objetivo é registrar informações detalhadas sobre instituições, cursos e corpo docente, proporcionando uma visão abrangente do sistema educacional superior. Esses dados, que combinam aspectos quantitativos e descritivos, são essenciais para promover transparência, embasar políticas públicas e orientar decisões estratégicas no setor educacional.

2.4.1 Dados Coletados

O Censo organiza suas informações em categorias principais:

- **Instituições de Ensino Superior (IES)**: Detalhes sobre universidades, centros universitários, faculdades, institutos federais e outras entidades, incluindo denominação, organização (ex.: universidade, centro universitário), categoria administrativa (ex.: pública federal, privada) e localização (município, estado, região).
- **Cursos**: Informações sobre programas de graduação, abrangendo modalidade (presencial ou a distância), tipo de grau (bacharelado, licenciatura, tecnólogo), área do conhecimento, duração do curso e quantidade de vagas oferecidas.
- **Docentes**: Perfil quantitativo do corpo docente, com dados sobre titulação (doutorado, mestrado, especialização), regime de trabalho (tempo integral ou parcial) e carga horária.

2.4.2 Disponibilização dos Dados

Os dados do Censo são disponibilizados em dois formatos principais:

- **Microdados**: Arquivos *CSV* organizados por ano, contendo extensos conjuntos de dados tabulares (Figura 6). Esses arquivos são ideais para especialistas que utilizam ferramentas como SQL, Python ou softwares estatísticos, mas sua complexidade restringe o acesso por usuários não técnicos.
- **Painel do Censo da Educação Superior**: Uma plataforma web interativa (Figura 7), acessível por meio de uma ferramenta de terceiros, que permite consultas visuais

Figura 6 – Os dados do censo são disponibilizados via CSV, com filtragem por ano

filtradas por ano, região, IES ou curso. Apesar de mais intuitiva, a interface apresenta limitações para consultas personalizadas ou detalhadas.

Figura 7 – O Painel disponibiliza uma interface mais amigável para consulta dos dados do Censo

O *Text-to-SQL* é uma solução prática para driblar essas dificuldades, conectando o esquema relacional do Censo da Educação Superior a sistemas conversacionais baseados em inteligência artificial. Com modelos de linguagem modernos, ele funciona como uma ponte, permitindo que usuários façam consultas dinâmicas e explorem dados em tempo real sem precisar de conhecimentos técnicos avançados.

2.5 Métricas de Avaliação do Text-to-SQL

A avaliação de sistemas Text-to-SQL requer métricas que analisem a precisão e eficiência das consultas SQL geradas a partir de linguagem natural. Essas métricas são fundamentais para assegurar que as consultas sejam sintaticamente corretas e produzam resultados confiáveis, sendo essenciais para aplicações em bancos de dados relacionais. Na sequência, apresenta-se uma explicação geral das métricas utilizadas neste trabalho. Para facilitar a compreensão, mais detalhes de cada métrica, acompanhados de exemplos, serão apresentados na seção de resultados, ficando assim mais próximos dos parágrafos que empregam as métricas.

2.5.1 Precisão de Correspondência Exata (EM)

A Precisão de Correspondência Exata (*Exact Matching Accuracy*, EM) é uma métrica que avalia se a consulta SQL gerada por um modelo é estruturalmente equivalente a uma consulta de referência (*ground-truth*). Diferentemente de métricas que avaliam o resultado da execução, a EM foca na lógica e na sintaxe da consulta em si (ZHU et al., 2024).

A fórmula geral para a métrica é:

$$\text{EM} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(\hat{Y}_i = Y_i)$$

onde:

- N: O número total de perguntas no conjunto de avaliação.
- $\hat{Y}_i$: A consulta SQL gerada pelo modelo para a i-ésima pergunta.
- Y_i: A consulta SQL de referência (correta) para a i-ésima pergunta.
- $\mathbb{I}(\cdot)$: Função indicadora. Neste trabalho, a condição $\hat{Y}_i = Y_i$ é considerada verdadeira (retornando 1) se as consultas forem estruturalmente equivalentes após um processo de padronização. Caso contrário, retorna 0.

2.5.2 Precisão de Execução (EX)

A Precisão de Execução (*Execution Accuracy*, EX) avalia se a consulta SQL gerada pelo modelo, quando executada no banco de dados, retorna o mesmo resultado que a consulta de referência. Diferentemente do EM, o EX foca na correção semântica, ignorando diferenças sintáticas, desde que o resultado seja correto (ZHU et al., 2024).

A fórmula do EX é:

$$\text{EX} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(f(Q_i, S_i) = A_i)$$

onde:

- N: Número total de consultas.
- Q_i: i-ésima pergunta em linguagem natural.
- S_i: Esquema do banco de dados correspondente à i-ésima pergunta.
- A_i: Resultado de referência (correto) para a i-ésima pergunta.
- $f(Q_i, S_i)$: Resultado retornado pela execução da consulta SQL gerada para Q_i no esquema S_i.
- $\mathbb{I}(\cdot)$: Função indicadora, que retorna 1 se $f(Q_i, S_i) = A_i$, e 0 caso contrário.

2.5.3 Pontuação de Eficiência Válida (VES)

A Pontuação de Eficiência Válida (*Valid Efficiency Score*, VES) avalia tanto a correção quanto a eficiência das consultas SQL geradas. Ela combina dois aspectos: (1) se a consulta é válida (executa corretamente e retorna o resultado esperado) e (2) quão eficiente é sua execução em comparação com a consulta de referência (ZHU et al., 2024).

A fórmula do VES é:

$$\text{VES} = \frac{1}{N} \sum_{i=1}^{N} \left(\mathbb{I}(Q_i^{\text{gen}} = Q_i^{\text{gold}}) \cdot \frac{T_{\text{gold}}}{T_{\text{gen}}} \right)$$

onde:

- N: Número total de consultas.
- Q_i^{gen}: Consulta SQL gerada para o i-ésimo exemplo.
- Q_i^{gold}: Consulta SQL de referência para o i-ésimo exemplo.
- $\mathbb{I}(\cdot)$: Função indicadora, que retorna 1 se Q_i^{gen} produz o mesmo resultado que Q_i^{gold}, e 0 caso contrário.
- T_{gold}: Tempo de execução da consulta de referência.
- T_{gen}: Tempo de execução da consulta gerada.

2.6 Trabalhos relacionados

A pesquisa em modelos Text-to-SQL tem sido impulsionada por grandes conjuntos de dados e *benchmarks* que desafiam a capacidade dos modelos de converter linguagem natural em consultas SQL. Um dos mais influentes é o Spider, introduzido por Yu et al. (2018), que fornece um conjunto de dados de grande escala para *parsing* semântico complexo e *cross-domain*. Mais recentemente, Li et al. (2023) apresentaram o BIRD, um *benchmark* para bancos de dados em larga escala, destacando desafios como conteúdos de banco de dados sujos e eficiência de SQL em cenários reais.

Qin et al. (2022) oferecem uma revisão abrangente sobre o *parsing* Text-to-SQL, categorizando corpora em *single-turn* e *multi-turn* e discutindo a transição para modelos de aprendizado profundo. Katsogiannis-Meimarakis e Koutrika (2023) complementa essa análise com uma taxonomia detalhada de sistemas neurais Text-to-SQL, enfatizando métodos baseados em aprendizado profundo e os desafios abertos, como a manipulação de consultas ambíguas e a escalabilidade para bancos de dados complexos.

O advento dos LLMs, como GPT-3, T5 (*Text-to-Text Transfer Transformer*) e modelos subsequentes, trouxe avanços significativos em Text-to-SQL, impulsionados por sua capacidade de compreender linguagem natural e gerar código estruturado. Shi et al. (2024) revisam o uso de LLMs em tarefas Text-to-SQL, abordando *datasets* de *benchmark* (como Spider, BIRD e DuSQL), técnicas de engenharia de *prompts* (como *few-shot learning* e *Retrieval-Augmented Generation*) e métodos de *fine-tuning*, como *LoRA*. Eles relatam que a precisão no *dataset Spider* aumentou de 73% para 91,2% com modelos como GPT-4, destacando o impacto dos LLMs na área. Além disso, Gao et al. (2025) introduziram o *XiYan-SQL*, um *framework* que utiliza uma estratégia que combina

vários modelos para melhorar a geração de candidatos em tarefas Text-to-SQL. O *XiYan-SQL* inclui o *M-Schema*, uma representação semi-estruturada de esquemas de banco de dados para melhorar a compreensão das estruturas de dados. O *framework* combina aprendizagem *in-context* com *fine-tuning* supervisionado para aprimorar a qualidade e a diversidade das consultas SQL geradas.

Zhu et al. (2024) classificam métodos aprimorados por LLMs em quatro categorias: engenharia de *prompts*, *fine-tuning*, pré-treinamento e abordagens baseadas em agentes. Eles também resumem métricas de avaliação, como *Exact Matching Accuracy, (EM)*, *Execution Accuracy (EX)* e *Valid Efficiency Score (VES)*, que são amplamente utilizadas para comparar o desempenho dos modelos. Além disso, Deng, Chen e Zhang (2022) oferecem uma visão geral das conquistas e expectativas futuras, destacando a importância de codificar significados de enunciados naturais e traduzir semântica entre linguagem natural e SQL.

Estudos específicos também avançam o campo. Pourreza e Rafiei (2023) introduzem o Din-SQL, uma abordagem de aprendizagem *in-context* decomposta que melhora a precisão por meio de autocorreção, alcançando resultados superiores em *benchmarks* como Spider e BIRD. Sun et al. (2023) apresentam o SQL-PALM, que otimiza a adaptação de LLMs para Text-to-SQL, demonstrando alta precisão em *datasets* como Spider e DuSQL. Tai et al. (2023) exploram o *prompting* do tipo *Chain of Thought*, que melhora o raciocínio em tarefas Text-to-SQL, com ganhos em modelos como T5 e GPT-3. Esses trabalhos fornecem um arcabouço teórico e prático para a avaliação de modelos Text-to-SQL baseados em LLMs.

Embora a maioria dos estudos em *Text-to-SQL* seja centrada no inglês, há esforços para expandir a tarefa para outros idiomas, incluindo o português. José e Cozman (2021) adaptaram o modelo RAT-SQL+GAP para o português, utilizando um modelo BART multilíngue e uma versão traduzida do conjunto de dados Spider, alcançando 83% do desempenho *baseline* em português. Por outro lado, Antoni et al. (2021) desenvolveu um sistema de perguntas e respostas baseado em Processamento de Linguagem Natural (PLN) para consultar os microdados do Censo Escolar Brasileiro, um conjunto de dados abertos com informações sobre escolas, alunos e matrículas da educação básica. O sistema utiliza uma abordagem híbrida que combina regras baseadas em palavras-chave com técnicas de PLN, por meio da API Google Natural Language. O protótipo permite consultas em linguagem natural, como " Onde é ofertado o curso de enfermagem em Santa Maria/RS", gerando consultas SQL automaticamente a partir dos dados do Censo Escolar de 2019. A validação, realizada com testes em perguntas conhecidas e um questionário com servidores do IFRS Campus Rolante, indicou que 81,8% consideraram a ferramenta útil para gestores e cidadãos. Contudo, o sistema enfrenta limitações, como erros no REN (e.g., nomes de escolas confundidos com pessoas) e ambiguidades em termos como "informática" versus "informática para internet", refletindo desafios do *Text-to-SQL* em português devido à complexidade do esquema de dados. Assim, o trabalho de Antoni et al. (2021) contribui para tornar os dados educacionais brasileiros mais acessíveis, complementando esforços como os de José e Cozman (2021) em um domínio de relevância social.

3 DESENVOLVIMENTO

Para assegurar uma avaliação equitativa dos diferentes modelos, foi criado um pipeline de validação unificado em Python, que é aplicado de forma idêntica a cada um. Este pipeline gerencia o ciclo completo de avaliação: ele submete a consulta SQL gerada por um modelo a uma API remota que acessa a base de dados real e, em seguida, utiliza um conjunto comum de funções para calcular as métricas de desempenho.

Esta seção detalha as tecnologias empregadas, a modelagem do esquema de dados, a arquitetura deste pipeline de avaliação, os processos de execução e validação das consultas, a aplicação aos microdados educacionais e o uso do ambiente Google Colab com GPU T4 para a condução dos experimentos.

3.1 Tecnologias Usadas

O experimento utiliza tecnologias escolhidas pela sua confiabilidade e adequação ao processamento dos dados do Censo da Educação Superior. A linguagem Python foi escolhida por sua sintaxe clara e suporte a bibliotecas. O ambiente inclui:

- **sqlparse**: Biblioteca para análise de consultas SQL, usada para decompor cláusulas (`SELECT`, `FROM`, `WHERE`) e comparar a estrutura das consultas geradas com as de referência, essencial para a métrica *Exact Matching* (EM).
- **json**: Biblioteca padrão para serializar e desserializar dados em requisições às APIs remotas.
- **os**: Biblioteca para acesso a variáveis de ambiente (via `os.getenv`), configurando parâmetros como URLs de APIs de forma segura e portátil.
- **gradio_client** (versão 0.7.1): Biblioteca para comunicação com APIs remotas dos modelos, como *XiYan-SQL*, via requisições HTTPS assíncronas.
- **ngrok**: Ferramenta que cria túneis HTTPS para expor a API intermediária local, permitindo acesso remoto pelo Google Colab.
- **bitsandbytes**: Biblioteca para quantização de modelos, reduzindo o consumo de VRAM e viabilizando a execução de modelos como *xiyan-qwen-instruct*, *OneSQL-v0.1-Qwen-7B*, *sqlcoder-7b-2*, e *OmniSQL-7B* na GPU T4 do Google Colab.
- **transformers, accelerate, peft**: Bibliotecas para carregar e executar modelos locais (*transformers*), otimizar inferência em hardware (*accelerate*), e realizar adaptação eficiente (*peft*), suportando tarefas como tokenização e validação semântica.

3.2 Uso do Google Colab com Ambiente T4

O experimento foi implementado no Google Colab, utilizando o ambiente de execução **T4** (*NVIDIA T4 GPU*), em razão de suas características técnicas e das limitações da conta gratuita do Colab, que restringem o acesso a GPUs mais potentes, como A100 ou V100, disponíveis apenas para usuários dos planos *Pro/Pro+*.

A GPU T4 oferece 16 GB de VRAM (GDDR6), 320 *Tensor Cores* e 2560 *CUDA Cores*, sendo suficiente para hospedar modelos de linguagem de médio porte, como *XiYan-SQL*, *OneSQL-v0.1-Qwen-7B*, *sqlcoder-7b-2* e *OmniSQL-7B*, quando utilizados com técnicas de otimização como quantização em 4 bits (`bnb-4bit`) e uso de `torch.float16`. O carregamento dos modelos, realizado via biblioteca `transformers` com `peft`, demanda uma gestão eficiente de memória, utilizando estratégias como `device_map=auto` para distribuir os pesos automaticamente entre GPU e CPU, além de tokenização otimizada e pipeline de geração de texto.

O ambiente também disponibiliza 12,7GB de RAM, 2 *vCPUs* (Intel Xeon, ∼2,2GHz) e armazenamento SSD variável (até 150 GB), sendo adequado para o fluxo de trabalho que envolve tanto execução local dos modelos quanto chamadas a APIs externas. Especificamente, o modelo *XiYan-SQL* é acessado via API remota, operando fora da infraestrutura local, o que dispensa uso da GPU para esse caso.

A escolha da GPU T4 se justifica principalmente pela necessidade de acomodar modelos de 7 bilhões de parâmetros quantizados, cujo carregamento integral exige praticamente toda a VRAM disponível da T4, além de parte da memória RAM. Embora mais potentes, GPUs como a A100 (40GB) ou V100 (16GB) não são acessíveis no plano gratuito, e o uso de ambientes *CPU-only* foi descartado devido à latência excessiva na inferência, particularmente na tokenização, geração e pós-processamento das consultas SQL. Portanto, a T4 representa um equilíbrio entre custo, disponibilidade e capacidade para este experimento.

3.3 Modelagem do Esquema Relacional

O esquema relacional foi codificado como uma *string* SQL (db_schema), definindo 18 tabelas que replicam a estrutura hierárquica modelada por Trevisan (2024), permitindo análises detalhadas de instituições, cursos, docentes e indicadores educacionais. A modelagem é essencial para superar a complexidade dos dados originais, frequentemente disponibilizados em arquivos *CSV* de difícil manipulação.

O código a seguir mostra como a tabela CENSO_IES é declarada:

```
# Schema completo do banco de dados
db_schema = """
CREATE TABLE CENSO_IES (
    COD_IES INT PRIMARY KEY,
    NOME_IES VARCHAR(255),
    SIGLA_IES VARCHAR(50),
    IN_CAPITAL INT CHECK (IN_CAPITAL IN (0, 1)),
    COD_MUNICIPIO VARCHAR(20),
    ID_CATEGORIA_ADMINISTRATIVA INT,
    ID_ORGANIZACAO_ACADEMICA INT,
    FOREIGN KEY (COD_MUNICIPIO) REFERENCES MUNICIPIOS_IBGE(COD_IBGE),
    FOREIGN KEY (ID_CATEGORIA_ADMINISTRATIVA) REFERENCES
       CENSO_CATEGORIAS_ADMINISTRATIVAS(ID_CATEGORIA_ADMINISTRATIVA),
    FOREIGN KEY (ID_ORGANIZACAO_ACADEMICA) REFERENCES
       CENSO_ORGANIZACOES_ACADEMICAS(ID_ORGANIZACAO_ACADEMICA)
);
"""
```

Bloco de código 1 – Esquema da tabela CENSO_IES

O esquema abrange tabelas como:

- **CENSO_IES**: Representa as instituições de ensino, com `COD_IES` como chave primária do tipo `INT` e `NOME_IES` do tipo `VARCHAR(255)`.
- **CENSO_CURSOS**: Representa os cursos, referenciando `COD_IES` como chave estrangeira.
- **DADOS_IES_DOCENTES**: Representa os docentes vinculados às instituições, utilizando uma chave composta formada por `NU_ANO_CENSO` e `CO_IES`.
- **MUNICIPIOS_IBGE** e **UF_IBGE**: Tabelas auxiliares contendo codificações do IBGE, com chaves alfanuméricas do tipo `VARCHAR(20)`.
- **CENSO_GRAUS_ACADEMICOS**: Tabela auxiliar para categorização das titulações acadêmicas.

As chaves estrangeiras formam um grafo relacional direcionado, representado internamente como uma lista de adjacência para verificação de integridade referencial.

3.4 Elaboração das Consultas de Referência (Gabarito)

A avaliação quantitativa do desempenho dos modelos exige uma base de comparação precisa, ou *ground truth*, que funciona como um gabarito (YU et al., 2018). Para este fim, um conjunto de perguntas foi construído manualmente com base no esquema relacional derivado do Censo da Educação Superior. Para cada pergunta, uma consulta SQL de referência correspondente foi elaborada.

Cada consulta de referência foi formulada para ser semanticamente correta, respondendo com precisão à intenção da pergunta. Elas representam a solução ideal para cada questão e servem como o padrão-ouro contra o qual as consultas geradas pelos modelos são rigorosamente avaliadas, considerando critérios como correção sintática, semântica e eficiência.

O papel da consulta de referência é central no pipeline de validação, servindo como base para as principais métricas de avaliação. A estrutura da consulta de referência é o alvo para o cálculo do *Exact Matching (EM)*; o resultado de sua execução é o gabarito para a *Execution Accuracy (EX)*; e seu tempo de execução é a linha de base para o *Valid Efficiency Score (VES)*.

Como exemplo, para uma pergunta específica do conjunto de testes:

> **Pergunta:** "A instituição universidade federal de pernambuco está localizada em uma cidade capital?"

A seguinte consulta de referência foi criada:

```
SELECT IN_CAPITAL FROM INEP.CENSO_IES WHERE NOME_IES = 'UNI
VERSIDADE FEDERAL DE PERNAMBUCO';
```

Esta consulta acessa de forma direta e eficiente a coluna booleana `IN_CAPITAL` para obter a resposta, representando a formulação ideal para a pergunta dada. A existência deste gabarito curado é, portanto, um pilar fundamental para a validade de toda a análise comparativa realizada neste trabalho.

3.5 Conexão e Execução via API Remota

Diferentemente de uma abordagem com bancos de teste simulados, a avaliação dos modelos neste trabalho utiliza uma conexão com uma base de dados real e persistente do Censo da Educação Superior, hospedada em um servidor institucional gerenciado pelo Centro de Processamento de Dados (CPD) da Universidade Federal de Santa Maria (UFSM). O acesso direto a essa base a partir do ambiente de execução (Google Colab) é inviabilizado pela necessidade de uma VPN institucional. Para contornar essa restrição, foi desenvolvida uma API intermediária utilizando o framework `Flask`, que atua como uma ponte segura entre os modelos de inteligência artificial e o banco de dados. A infraestrutura de dados foi disponibilizada pelo CPD da UFSM, que manteve o banco acessível de forma segura e controlada.

A API intermediária, implementada em Python, expõe um endpoint `/query` que aceita consultas SQL via método GET, executa-as diretamente no banco de dados DB2 utilizando a biblioteca `ibm_db`, conforme ilustra o Bloco de código 2, e retorna os resultados em formato JSON. A conexão com o banco é estabelecida por meio de uma string de conexão configurada com parâmetros padrão, incluindo o nome do banco, endereço do servidor, porta, protocolo TCP/IP, e credenciais de autenticação, assegurando acesso seguro ao banco hospedado pelo CPD da UFSM. Para viabilizar o acesso remoto a partir do Google Colab, foi utilizado o `ngrok`.

```
@app.route('/')
def home():
    """API para acesso ao banco de dados está funcionando! Use /query?
       sql=SUA_QUERY_AQUI"""
app.route("/api", methods=["GET"])
def query_db():
    sql_query = request.args.get("sql")
    if not sql_query:
        return jsonify({"error": "Nenhuma query SQL fornecida. Use o
           parâmetro ?sql=SUA_QUERY na URL."}), 400
    print(f"Recebida query SQL: {sql_query}")
    conn = None
    try:
        conn = ibm_db.connect(DB_CONN_STR, "", "")
        print("Conexão com o banco de dados bem-sucedida!")
        stmt = ibm_db.exec_immediate(conn, sql_query)
        # result_list = []
        # continuação do código
    except:
        return None, 0.0
```

Bloco de código 2 – API intermediária

A comunicação com o endpoint `/query` é gerenciada pela função `execute_query_on_real_db`, ilustrada no Bloco de código 3. Esta função codifica a consulta SQL para transmissão segura, mede o tempo de execução (*wall-clock time*) para avaliar a eficiência, e trata as respostas JSON, capturando tanto resultados válidos quanto erros.

```
def execute_query_on_real_db(sql_query: str) -> Tuple[optional[Dict],
   float]:
    """Executa a query SQL no backend via ngrok e retorna o resultado
       e o tempo de execução."""
    if not sql_query:
```

```
        print("Query SQL vazia. Abortando execução.")
        return None, 0.0
    query_codificada = urllib.parse.quote_plus(sql_query)
    url_api = f"{NGROK_URL}/query?sql={query_codificada}"
    try:
        start_time = time.time()
        response = requests.get(url_api, headers={"ngrok-skip-browser-
            warning": "true"}, timeout=30)
        response.raise_for_status()
        end_time = time.time()
        dados = response.json()
        return dados, end_time - start_time
    except requests.exceptions.RequestException as e:
        print("Erro ao executar a query no banco:")
        print(f"Erro: {e}")
        if hasattr(e, 'response') and e.response:
            print(f"Resposta bruta: {e.response.text}")
        return None, 0.0
```

Bloco de código 3 – Função execute_query_on_real_db

A função `execute_query_on_real_db` implementa um fluxo de execução bem estruturado para facilitar a correta avaliação de cada consulta gerada:

- **Codificação da URL:** A consulta SQL recebida é processada pela função `urllib.parse.quote_plus`. Isso viabiliza a conversão de caracteres especiais na string SQL (como espaços, aspas ou operadores) sejam convertidos para um formato seguro para transmissão em uma URL, evitando erros de interpretação pelo servidor da API.

- **Medição de Tempo:** O tempo de execução é medido utilizando `time.time()` antes e depois da chamada à API. Essa abordagem captura o *wall-clock time*, que inclui não apenas o tempo de processamento da consulta no banco de dados, mas também a latência da rede. Isso fornece uma métrica de eficiência mais realista, que é fundamental para o cálculo do *Valid Efficiency Score (VES)*.

- **Tratamento de Resposta e Erros:** A função utiliza a biblioteca `requests` para realizar a chamada HTTP. A resposta é desserializada de JSON para um dicionário Python. A utilização de `response.raise_for_status()` juntamente com o bloco `try...except` permite que falhas de comunicação ou erros retornados pelo servidor (como um status HTTP 500) sejam devidamente capturados.

Como exemplo, uma consulta para obter a sigla de uma instituição específica:

```
{SELECT SIGLA_IES FROM CENSO_IES WHERE NOME_IES = 'Universida
de X';}
```

resultaria em uma chamada à API e, em caso de sucesso, o retorno de um objeto JSON estruturado da seguinte forma:

```
{"columns": ["SIGLA_IES"], "data": [{"SIGLA_IES": "X"}], "query": "
    SELECT SIGLA_IES ..." }
```

Este formato detalhado, que inclui os nomes das colunas e a própria consulta executada, facilita a validação e a depuração. Caso a consulta contenha um erro de sintaxe, a API

retornaria uma estrutura similar, mas com o campo `data` vazio e uma chave `error` descrevendo a falha, permitindo o registro preciso do erro de geração do modelo.

3.6 Análise Estrutural da Consulta SQL

A primeira etapa do pipeline de validação consiste na análise estrutural da consulta SQL gerada. Esse processo é fundamental para verificar se o modelo formulou uma consulta logicamente correta, antes mesmo de sua execução, e serve de base para a métrica *Exact Matching (EM)*.

A função `normalize_sql` é responsável por essa padronização inicial. Ela processa a string SQL bruta para remover variações sintáticas que não impactam a semântica da consulta. O tratamento inclui a remoção de comentários, cláusulas `ORDER BY`, aliases de tabelas e colunas, além da conversão do texto para um formato canônico em minúsculas. O código a seguir mostra a função:

```
def normalize_sql(sql: str) -> str:
    """Normaliza uma consulta SQL para comparacao, removendo aliases e
        ORDER BY."""
    # Remover comentários
    sql = re.sub(r'--.*?$|/\*?.*?\*/', '', sql, flags=re.DOTALL)
    # Converter para minúsculas
    sql = sql.lower().strip()
    # Remover ORDER BY
    sql = re.sub(r'\s*order\s+by\s+.*?(?={|$)', '', sql, flags=re.
       IGNORECASE)
    # Remover aliases (AS nome)
    sql = re.sub(r'\s+as\s+\w+', '', sql, flags=re.IGNORECASE)
    # Normalizar espaços
    sql = re.sub(r'\s+', ' ', sql)
    sql = re.sub(r"like\s+%\(['%']+\)%", r"'\1'", sql, flags=re.
       IGNORECASE)
    # Garantir ponto e vírgula no final
    if not sql.endswith(';'):
        sql += ';'
    return sql.strip()
```

Bloco de código 4 – Função normalize_sql

Uma vez normalizada, a consulta é decomposta em seus componentes lógicos (`SELECT`, `FROM`, `JOIN` e `WHERE`) pela biblioteca `sqlparse` e por funções auxiliares. Por exemplo, para uma consulta que busca o nome de uma instituição pela sua sigla, o resultado da decomposição seria:

```
=== Componentes Normalizados ===
SELECT Columns: ['nome_ies']
Tables: ['censo_ies']
Joins: []
WHERE Conditions: ["sigla_ies = 'farec'"]
```

A comparação desses componentes com os de uma consulta de referência é então realizada pela função `calculate_exact_matching_accuracy`. Ela verifica a igualdade entre cada um dos quatro conjuntos de componentes. No exemplo, como todos os componentes correspondem perfeitamente aos da referência, a consulta alcança o score máximo de 1.0 para a métrica *Exact Matching (EM)*.

3.7 Padronização dos Resultados

Após a execução da consulta pela API remota, os resultados retornados em formato JSON são submetidos a um segundo processo de padronização. Este passo é crucial para o cálculo da métrica *Execution Accuracy (EX)*, pois faz com que a comparação a comparação entre os dados gerados e os de referência seja consistente e imune a variações de formatação.

A função `compare_sql_results` realiza esta tarefa. Diferentemente de uma simples ordenação, ela trata os resultados como um multiconjunto de valores. Na prática, a função normaliza o conteúdo textual (para minúsculas e sem espaços extras) e desconsidera a ordem em que as colunas e as linhas são retornadas pelo banco de dados, focando exclusivamente no conteúdo semântico. O código a seguir mostra a função:

```
def compare_sql_results(generated_data: list, reference_data: list) ->
    Tuple[bool, float]:
    """Compara os valores dos resultados, ignorando nomes de colunas e
        normalizando valores,
    independentemente da ordem das linhas."""
    if len(generated_data) != len(reference_data):
        print(f"Debug: Número de linhas diferente (Gerado: {len(
            generated_data)}, Referência: {len(reference_data)})")
        return False, 0.0

    # Normalize values
    def normalize_value(v):
        if isinstance(v, str):
            return v.lower().strip()
        elif isinstance(v, (int, float)):
            return v
        else:
            return str(v).lower().strip()

    # Continue
```

Bloco de código 5 – Função compare_sql_results

Por exemplo, para a pergunta "Quantas instituições existem na cidade de Campinas?", a API retorna um resultado numérico. A função `compare_sql_results` processa este retorno e o converte em uma representação padronizada, que separa os tipos de dados:

```
Valores Normalizados: Strings: set(), Numbers: {23}
```

Esta representação padronizada dos resultados é então utilizada pela função `calculate_execution_accuracy` para determinar a correção semântica da consulta. A função compara o conjunto de valores normalizados da consulta do modelo com o da consulta de referência. Se os conjuntos forem idênticos, a função retorna um score de 1.0, validando que a consulta gerada recuperou os dados corretos.

3.8 Análise de Eficiência por Tempo de Execução

A avaliação da eficiência computacional das consultas geradas é realizada através de uma abordagem prática e realista: a medição do tempo de execução real (*wall-clock time*). Esta medição quantifica o desempenho da consulta em um cenário real, incluindo

o tempo de processamento no banco de dados e a latência da rede da API. A métrica utilizada para essa análise é a *Valid Efficiency Score (VES)*.

O cálculo do VES, implementado na função `calculate_valid_efficiency_score`, ocorre em duas etapas. Primeiramente, a função verifica se a consulta gerada é "válida", ou seja, se o seu resultado corresponde ao da consulta de referência (conforme medido pela métrica *Execution Accuracy*). Uma consulta que retorna dados incorretos, por mais rápida que seja, recebe um score VES de 0. O código a seguir mostra a função:

```
def calculate_valid_efficiency_score(generated_result: optional[Dict],
     reference_result: optional[Dict], generated_time: float,
    reference_time: float) -> float:
"""Calcula o Valid Efficiency Score para uma query."""
if generated_result is None or reference_result is None or
    generated_time == 0.0:
return 0.0
exact_match, partial_score = compare_sql_results(generated_result.get(
    "data", []), reference_result.get("data", []))
if (exact_match_or_partial_score > 0) and reference_time > 0:
print(f"Debug VES: generated_time={generated_time}, reference_time={
    reference_time}")
return reference_time / generated_time
return 0.0
```

Bloco de código 6 – Função calculate_valid_efficiency_score

Se a consulta for válida, o score de eficiência é então calculado como a razão entre o tempo de execução da consulta de referência (T_{ref}) e o tempo da consulta gerada (T_{gen}), conforme a fórmula:

$$\text{VES} = \frac{T_{ref}}{T_{gen}}$$

Por exemplo, para uma consulta validada como correta, cujos tempos de execução registrados foram $T_{ref} = 0.8314s$ para a referência e $T_{gen} = 0.8302s$ para a gerada, o score VES resultante seria:

$$\text{VES} = \frac{0.8314}{0.8302} \approx 1.001$$

Um score como este, próximo a 1.0, indica que a consulta gerada teve um desempenho de eficiência quase idêntico ao da referência. Scores maiores que 1.0 indicam que a consulta gerada foi mais rápida, enquanto scores menores que 1.0 apontam para uma execução mais lenta.

Dessa forma, a análise de eficiência com o VES é crucial no contexto do Censo. Não basta que uma consulta esteja correta; ela também precisa ser rápida. A capacidade do modelo de gerar consultas eficientes (com VES próximo ou maior que 1.0) é o que determina se uma aplicação interativa sobre grandes volumes de dados é viável na prática, já que tempos de resposta longos comprometem diretamente a usabilidade do sistema.

3.9 Conjunto de Perguntas para Avaliação

Para avaliar de forma sistemática o desempenho dos modelos, foi construído manualmente um conjunto de 24 perguntas em português, com base no esquema relacional do Censo da Educação Superior. As perguntas foram projetadas para cobrir diferentes tipos de operações SQL (como seleção, agregação e junção) e foram classificadas em três níveis de complexidade: Fácil, Médio e Difícil.

Tabela 1 – Questões de Baixa Complexidade

Nº	Dificuldade	Pergunta
1	Fácil	Qual a sigla da instituição Faculdade Quirinópolis?
2	Fácil	Qual o nome da instituição de sigla Farec?
3	Fácil	Quantas instituições estão localizadas em cidades capitais?
4	Fácil	A instituição Universidade Federal de Pernambuco está localizada em uma cidade capital?
5	Fácil	Quantos cursos possuem tempo de integralização de 4 anos?

As questões de baixa complexidade, em geral, podem ser respondidas consultando apenas uma tabela, envolvendo poucas ou uma coluna na seleção e filtragem e não necessitando de manobras complexas de SQL.

Tabela 2 – Questões de Média Complexidade (Parte 1)

Nº	Dificuldade	Pergunta
1	Média	Quantas instituições existem na cidade de Campinas?
2	Média	Quantas instituições existem no estado de Minas Gerais?
3	Média	Quantas instituições existem na região Nordeste?
4	Média	Qual a distribuição de instituições por região?
5	Média	Qual a média de instituições entre as regiões?
6	Média	Quantas instituições de nível superior são de administração pública e quantas são privadas?
7	Média	Quantos cursos são ofertados pela instituição Faculdade de Rondônia?
8	Média	O curso Engenharia Civil é ofertado pela instituição Faculdade Sergipana?

Em contraste, as questões de média complexidade obrigam o modelo a consultar mais de uma tabela, sendo necessárias uma ou mais junções com chaves estrangeiras e envolvendo algumas colunas na elaboração das cláusulas de seleção e filtragem, mas não necessitam de manobras complexas de SQL.

As questões de alta complexidade projetadas fazem com que o modelo precise consultar diversas tabelas, coordenando um grande número de junções com chaves estrangeiras e envolvendo diversas colunas na elaboração das cláusulas de seleção e filtragem.

3.9.1 Integração dos Modelos com o Pipeline

Os modelos são integrados ao pipeline para processar as 24 perguntas do conjunto de teste, gerando consultas SQL que interagem com o esquema relacional do Censo da

Tabela 3 – Questões de Média Complexidade (Parte 2)

Nº	Dificuldade	Pergunta
9	Média	Quantos cursos de grau bacharelado são ofertados pela instituição Faculdade Cuiabá?
10	Média	Quantos cursos da área do conhecimento Ciências Sociais Aplicadas são ofertados pela instituição Faculdade Vitoriana de Tecnologia?
11	Média	Quantos cursos são ofertados pela modalidade curso a distância pela instituição Faculdade Taboão?
12	Média	Qual a média do tempo de integralização dos cursos da instituição Faculdade Fidelis?
13	Média	Qual a média do tempo de integralização dos cursos da área do conhecimento Ciências Exatas e da Terra?
14	Média	Quantos docentes na instituição Universidade do Estado do Rio de Janeiro têm título de doutorado?
15	Média	Quantos docentes na instituição Universidade da Amazônia têm título de mestrado ou especialização?

Tabela 4 – Questões de Alta Complexidade

Nº	Dificuldade	Pergunta
1	Difícil	Qual a distribuição de instituições, por organização acadêmica, por região IBGE?
2	Difícil	Qual a distribuição de instituições, por categoria administrativa, por estado?
3	Difícil	Qual a distribuição de cursos, por área do conhecimento, por região IBGE?
4	Difícil	Qual o percentual de docentes com doutorado, por instituição, por região IBGE?

Educação Superior, prefixado com `INEP.`. O fluxo, ilustrado na Figura 8, inicia com a função `generate_sql_from_query`, que combina a pergunta com o esquema do banco.

A consulta gerada é normalizada por `normalize_sql`, que extrai componentes estruturais para o cálculo do EM via `calculate_exact_matching_accuracy`. A consulta é executada no banco real via `execute_query_on_real_db`, que trata falhas HTTP ou queries vazias, retorna resultados JSON, e registra o tempo de execução. Os resultados são padronizados por `compare_sql_results`, permitindo o cálculo do EX com `calculate_execution_accuracy`. O VES é calculado por `calculate_valid_efficiency_score`, comparando tempos de execução. Métricas individuais (EM, EX, VES) são armazenadas em listas para análise agregada por nível de dificuldade, possibilitando uma avaliação detalhada da precisão sintática, correção semântica, e eficiência computacional.

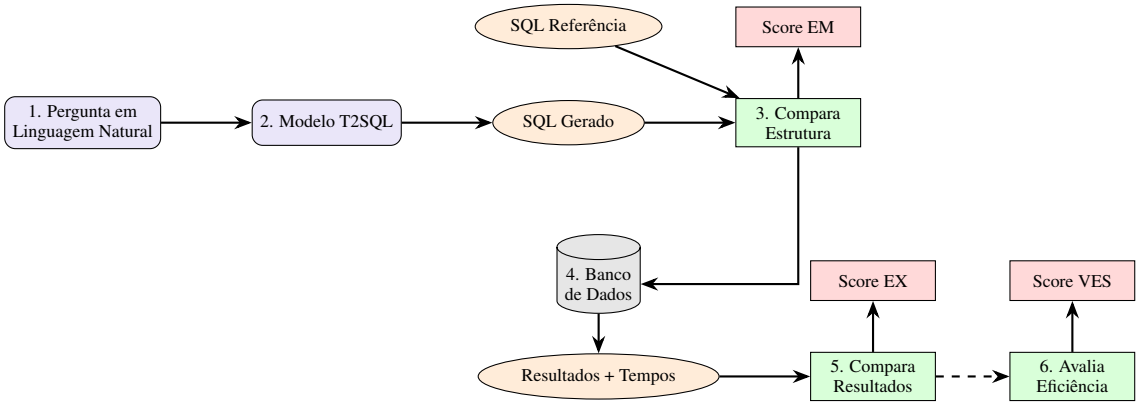

Figura 8 – Fluxo da avaliação das métricas.

3.9.2 Funcionamento do Pipeline em Casos Práticos

Para ilustrar o funcionamento do pipeline de validação, considera-se uma pergunta típica de média complexidade, como "Quantas instituições de ensino superior são públicas e quantas são privadas?". Esta pergunta requer uma consulta SQL com junções e agregações, refletindo cenários comuns no conjunto de teste (Tabela 2).

1. **Geração da Consulta SQL:** A função `generate_sql_from_query` traduz a pergunta em uma consulta SQL, utilizando o esquema do banco com prefixo `INEP.`. Por exemplo, a consulta gerada poderia ser:

```
SELECT COUNT(*) AS total,
       SUM(CASE WHEN cca.ID_CATEGORIA_ADMINISTRATIVA = 1 THEN 1
           ELSE 0 END) AS publicas,
       SUM(CASE WHEN cca.ID_CATEGORIA_ADMINISTRATIVA = 2 THEN 1
           ELSE 0 END) AS privadas
FROM INEP.CENSO_IES ci
JOIN INEP.CENSO_CATEGORIAS_ADMINISTRATIVAS cca
ON ci.ID_CATEGORIA_ADMINISTRATIVA = cca.
   ID_CATEGORIA_ADMINISTRATIVA;
```

   enquanto a consulta de referência seria:

```
SELECT SUM(CASE WHEN cat_adm.ID_CATEGORIA_ADMINISTRATIVA = 1 THEN
   1 ELSE 0 END) AS publicas,
       SUM(CASE WHEN cat_adm.ID_CATEGORIA_ADMINISTRATIVA = 2 THEN
           1 ELSE 0 END) AS privadas
FROM INEP.CENSO_IES inst
JOIN INEP.CENSO_CATEGORIAS_ADMINISTRATIVAS cat_adm
ON inst.ID_CATEGORIA_ADMINISTRATIVA = cat_adm.
   ID_CATEGORIA_ADMINISTRATIVA;
```

   A consulta gerada inclui uma coluna extra (`COUNT(*)`), mas mantém a mesma lógica de junção e agregação.

2. **Normalização e Decomposição:** A função `normalize_sql` remove aliases, comentários, e cláusulas `ORDER BY`, converte para minúsculas, e normaliza funções como `SUM(CASE...)` com `normalize_sum_case`. A biblioteca `sqlparse` decompõe a consulta em componentes (`SELECT`, `FROM`, `JOIN`, `WHERE`). Para o exemplo:

```
=== Componentes Normalizados (SQL Gerado) ===
SELECT Columns: ['count(*)', 'sum(case id_categoria_administrativa
    = 1)', 'sum(case id_categoria_administrativa = 2)']
Tables: ['censo_categorias_administrativas', 'censo_ies']
Joins: ['censo_categorias_administrativas ON
   id_categoria_administrativa = id_categoria_administrativa']
WHERE Conditions: []
```

```
=== Componentes Normalizados (SQL Referência) ===
SELECT Columns: ['sum(case id_categoria_administrativa = 1)', 'sum
   (case id_categoria_administrativa = 2)']
Tables: ['censo_categorias_administrativas', 'censo_ies']
Joins: ['censo_categorias_administrativas ON
   id_categoria_administrativa = id_categoria_administrativa']
WHERE Conditions: []
```

As funções `extract_select_columns`, `extract_tables`, `extract_joins`, e `extract_where_conditions` garantem que aliases de tabelas (e.g., `ci`, `ies`) sejam ignorados.

3. **Execução via API:** A função `execute_query_on_real_db` codifica a consulta com `urllib.parse.quote_plus`, envia-a ao endpoint `/query` via `requests`, e mede o *wall-clock time*. Para o exemplo, suponha que a consulta gerada retorna:

```
{"columns": ["total", "publicas", "privadas"],
 "data": [{"total": 3575, "publicas": 121, "privadas": 151}],
 "query": "..."}
```

e a referência retorna:

```
{"columns": ["publicas", "privadas"],
 "data": [{"publicas": 121, "privadas": 151}],
 "query": "..."}
```

A consulta gerada é executada em $T_{gen} = 0,8273s$, e a referência em $T_{ref} = 0,8504s$. Erros (e.g., HTTP 500) são capturados via `try...except`.

4. **Cálculo de EM:** A função `calculate_exact_matching_accuracy` compara os componentes normalizados. No exemplo, `Tables`, `Joins`, e `WHERE` coincidem (score 1,0 cada), mas `SELECT` tem correspondência parcial (2/3 colunas coincidem, score $0,5 \times 2/3 = 0,333$). O score final é:

$$\text{EM} = \frac{1 + 1 + 1 + 0,333}{4} \approx 0,83$$

Isso reflete uma consulta estruturalmente correta, mas com uma coluna extra.

5. **Cálculo de EX:** A função `compare_sql_results` normaliza os resultados, separando strings e números:

```
Gerado:  Strings:  set(), Numbers:  {121, 151, 3575}
Referência:  Strings:  set(), Numbers:  {121, 151}
```

Como os números da referência são um subconjunto dos gerados, a correspondência é parcial, resultando em EX = 0,5. Um score de 1,0 exigiria igualdade exata.

6. **Cálculo de VES:** A função `calculate_valid_efficiency_score` verifica a validade via `compare_sql_results`. Como há correspondência parcial (EX = 0,5), calcula:

$$\text{VES} = \frac{T_{ref}}{T_{gen}} = \frac{0,8504}{0,8273} \approx 1,028$$

Um VES > 1,0 indica que a consulta gerada foi ligeiramente mais rápida, apesar da coluna extra.

Este processo, executado iterativamente para cada uma das 24 perguntas (Tabelas 1, 2, 3, 4), gera métricas individuais que são agregadas para análise por nível de dificuldade, possibilitando uma análise detalhada do desempenho dos modelos no contexto do Censo da Educação Superior, com ênfase na precisão sintática, correção semântica e eficiência computacional.

4 RESULTADOS

Esta seção apresenta os resultados da avaliação de quatro modelos de inteligência artificial para geração de consultas SQL no esquema do Censo da Educação Superior (prefixado com `INEP.`): *XiYan-SQL*, *OneSQL-v0.1-Qwen-7B*, *sqlcoder-7b-2* e *OmniSQL-7B*. Os resultados são organizados por nível de dificuldade das perguntas (Fácil, Médio e Difícil), apresentados em tabelas consolidadas com os valores de EM, EX e VES para cada modelo e consulta, seguidas de análises detalhadas. A seção conclui com uma síntese geral dos desempenhos observados.

4.1 Métricas

As métricas utilizadas para avaliar o desempenho dos quatro modelos de Text-to-SQL analisados, Exact Matching Accuracy (EM), Execution Accuracy (EX) e Valid Efficiency Score (VES), são detalhadas a seguir. Cada métrica é definida, contextualizada em relação ao Censo da Educação Superior e ilustrada com um exemplo prático baseado no esquema relacional do banco de dados INEP.

4.1.1 Precisão de Correspondência Exata (EM)

A Precisão de Correspondência Exata (*Exact Matching Accuracy*, EM) avalia se a consulta SQL gerada por um modelo é estruturalmente idêntica à consulta de referência (*ground-truth*), focando na equivalência sintática e lógica.

4.1.1.1 Detalhamento

A métrica EM verifica a equivalência entre a consulta gerada e a de referência, considerando componentes como cláusulas `SELECT`, `FROM`, `JOIN`, `WHERE` e `ORDER BY`. Um processo de normalização, implementado pela função `normalize_sql`, elimina diferenças irrelevantes, como aliases de tabelas, espaços extras e capitalização. A consulta gerada recebe um score EM de 1,0 se for idêntica à referência após normalização; caso contrário, o score é reduzido proporcionalmente às diferenças estruturais. Essa métrica

é essencial para avaliar a capacidade do modelo de compreender a gramática SQL e o esquema do banco de dados.

Cada componente é comparado separadamente, e a pontuação total é a média das pontuações individuais. Se um componente for idêntico ao da referência, contribui com 1,0 para o score; se não for idêntico, mas houver correspondência parcial (calculada pela interseção de elementos, como colunas ou condições), um score parcial é atribuído usando a fórmula 0.5 * (len(matching_elements) / len(ref_elements)). O peso 0.5 garante que correspondências parciais sejam menos valiosas que correspondências exatas, enquanto a proporção reflete o grau de similaridade. Por exemplo, se a consulta gerada acerta 2 de 3 colunas no SELECT, o score parcial para esse componente será 0.5 * (2/3) = 0.333. Se nenhum elemento corresponde, o score do componente é 0. Essa abordagem granular permite avaliar o modelo mesmo quando a consulta não é totalmente correta, capturando melhorias incrementais em tarefas complexas de Text-to-SQL.

4.1.1.2 Exemplo Prático

Considere a pergunta "Qual a sigla da instituição Faculdade Quirinópolis?" com base no esquema INEP, que inclui a tabela `CENSO_IES (ID_IES, NOME, SIGLA)`. A consulta de referência é:

```
SELECT SIGLA
FROM INEP.CENSO_IES
WHERE NOME = 'Faculdade Quirinôpolis';
```

Se o modelo gerar:

```
SELECT sigla
FROM INEP.CENSO_IES ies
WHERE nome = 'Faculdade Quirinôpolis';
```

Após normalização, ambas as consultas apresentam os mesmos componentes: `SELECT: [sigla],Tables:[censo_ies],WHERE: [nome = 'Faculdade Quirinópolis'],Joins:[]`. Como são equivalentes em todos os componentes analisados (SELECT, Tables, Joins e WHERE), o score EM é 1,0. Se a consulta gerada incluísse uma coluna adicional no `SELECT` (ex.: [sigla, nome]), ou uma condição diferente no `WHERE` (ex.: [nome = 'Outra Instituição']), o score seria reduzido, por exemplo, para 0,5, devido à falta de correspondência total nos componentes estruturais.

4.1.2 Precisão de Execução (EX)

A Precisão de Execução (*Execution Accuracy*, EX) avalia a correção semântica da consulta gerada, verificando se os resultados retornados pelo banco de dados são idênticos aos da consulta de referência, independentemente de diferenças sintáticas.

4.1.2.1 Detalhamento

A métrica EX utiliza a função `compare_sql_results` para normalizar e comparar os resultados das consultas, separando strings e números. Um score EX de 1,0 é atribuído quando os resultados são idênticos; um score de 0,0 é dado se os resultados diferem ou se a consulta é inválida. Essa métrica é crucial para garantir que o modelo mapeie

corretamente a intenção da pergunta em linguagem natural para o esquema relacional, sendo mais alinhada a aplicações práticas onde o resultado correto é prioritário.

Na implementação da função `compare_sql_results`, os resultados das consultas são normalizados para ignorar diferenças irrelevantes, como capitalização em strings ou ordem das linhas. Cada linha é dividida em conjuntos de strings e números, que são comparados como conjuntos para desconsiderar a ordem. Um score EX de 1,0 é atribuído se os conjuntos de resultados forem idênticos. Para correspondências parciais, a função verifica se todas as linhas geradas correspondem a linhas de referência, considerando subconjuntos de números ou proximidade numérica (dentro de 10% de diferença relativa para casos com uma única coluna numérica). Nesse caso, um score parcial de 0,5 é atribuído, reconhecendo que os resultados são semanticamente próximos, mas não idênticos. Por exemplo, se a consulta gerada retorna uma linha com Strings: set(), Numbers: 121, 151, 3575 e a referência retorna Strings: set(), Numbers: 121, 151, a função detecta que os números da referência são um subconjunto dos gerados, resultando em um score parcial de 0,5, pois a correspondência é parcial, mas significativa. Se o número de linhas diferir ou nenhuma linha corresponder, o score é 0,0. Essa abordagem permite avaliar a utilidade prática da consulta, mesmo em casos de diferenças sintáticas ou erros menores, sendo especialmente relevante para aplicações onde a exatidão dos resultados é mais importante que a estrutura da query.

4.1.2.2 Exemplo Prático

Para a pergunta "Quantas instituições estão localizadas em cidades capitais?", a consulta de referência é:

```
SELECT COUNT(*)
FROM INEP.CENSO_IES
WHERE CAPITAL = 1;
```

Se o modelo gerar:

```
SELECT COUNT(ID_IES)
FROM INEP.CENSO_IES
WHERE CAPITAL = 1;
```

Ambas as consultas retornam o mesmo resultado, por exemplo, `{357}`, resultando em um score EX de 1,0. No entanto, se o modelo gerasse `WHERE CAPITAL = 'Sim'`, a consulta seria inválida (pois `CAPITAL` é numérico), resultando em um score EX de 0,0.

4.1.3 Pontuação de Eficiência Válida (VES)

A Pontuação de Eficiência Válida (*Valid Efficiency Score*, VES) avalia a eficiência computacional da consulta gerada em relação à consulta de referência, considerando o tempo de execução.

4.1.3.1 Detalhamento

A métrica VES é calculada apenas para consultas válidas (com EX maior que 0,0), comparando o tempo de execução da consulta gerada (T_{gen}) com o da consulta de referência (T_{ref}). Um VES próximo de 1,0 indica eficiência semelhante; valores maiores que 1,0

sugerem maior rapidez, enquanto valores menores indicam lentidão. Essa métrica é vital para aplicações interativas com grandes volumes de dados, como o Censo da Educação Superior, onde tempos de resposta rápidos são essenciais.

4.1.3.2 Exemplo Prático

Para a pergunta "Quantos cursos possuem tempo de integralização de 4 anos?", a consulta de referência é:

```
SELECT COUNT(*)
FROM INEP.CENSO_CURSO
WHERE TEMPO_INTEGRALIZACAO = 4;
```

Se o modelo gerar a mesma consulta, com tempos de execução $T_{\text{ref}} = 0,8314\,\text{s}$ e $T_{\text{gen}} = 0,8302\,\text{s}$, o VES é aproximadamente 1,0, indicando eficiência semelhante. Se a consulta gerada incluísse uma junção desnecessária, aumentando T_{gen} para 1,2 s, o VES seria aproximadamente 0,69, refletindo menor eficiência.

4.2 Avaliação de perguntas

Os resultados da avaliação dos quatro modelos de Text-to-SQL são apresentados a seguir, com ênfase no desempenho em perguntas de diferentes níveis de complexidade. A análise está organizada por nível de dificuldade.

4.2.1 Nível Fácil

Os resultados das consultas de baixa complexidade, que testam a geração de consultas SQL precisas e eficientes, estão consolidados na Tabela 5. Esses dados permitem avaliar a robustez inicial dos modelos no esquema INEP.

Tabela 5 – Resultados das Consultas de Nível Fácil

		XiYan-SQL			**OneSQL-v0.1-Qwen-7B**			**sqlcoder-7b-2**			**OmniSQL-7B**		
Consulta	Descrição	EM	EX	VES	EM	EX	VES	EM	EX	VES	EM	EX	VES
1	Sigla da Faculdade Quirinópolis	1,00	1,00	0,9116	1,00	1,00	0,9409	0,75	0,00	0,0000	1,00	1,00	1,0222
2	Nome da instituição de sigla FAREC	1,00	1,00	0,9772	1,00	1,00	1,0590	1,00	1,00	0,9599	1,00	1,00	0,9260
3	Instituições em cidades capitais	1,00	1,00	0,9571	1,00	1,00	0,9048	0,50	0,00	0,0000	1,00	1,00	0,9844
4	UFPE em cidade capital	0,75	0,00	0,0000	0,75	0,00	0,0000	0,00	0,00	0,0000	1,00	1,00	0,9726
5	Cursos com integralização de 4 anos	1,00	1,00	0,9554	1,00	1,00	0,9673	1,00	1,00	1,0098	1,00	1,00	0,9125
Média		**0,950**	**0,800**	**0,7603**	**0,950**	**0,800**	**0,7744**	**0,650**	**0,400**	**0,3939**	**1,000**	**1,000**	**0,9635**

No nível Fácil, o *OmniSQL-7B* obteve o melhor desempenho geral, com médias de EM = 1,000, EX = 1,000 e VES = 0,9635, alcançando precisão perfeita em todas as consultas. O *XiYan-SQL* e o *OneSQL-v0.1-Qwen-7B* apresentaram desempenhos próximos, com EM = 0,950 e EX = 0,800, mas foram penalizados na consulta sobre a UFPE em cidade capital (EM = 0,75, EX = 0,00), devido a erros na identificação da condição. O *sqlcoder-7b-2* teve o pior desempenho, com médias de EM = 0,650, EX = 0,400 e VES = 0,3939, especialmente nas consultas sobre instituições em cidades capitais (EM = 0,50, EX = 0,00) e UFPE em cidade capital (EM = 0,00, EX = 0,00). O *OmniSQL-7B* destacou-se pela eficiência, com VES = 1,0222 na consulta sobre a sigla da Faculdade Quirinópolis, enquanto o *OneSQL-v0.1-Qwen-7B* obteve o maior VES individual (1,0590) na consulta sobre o nome da instituição FAREC.

4.2.2 Nível Médio

As consultas de complexidade intermediária, que envolvem junções e condições mais elaboradas, têm seus desempenhos sumarizados na Tabela 6. Esses resultados mostram como os modelos lidam com cenários moderadamente desafiadores do esquema INEP.

Tabela 6 – Resultados das Consultas de Nível Médio

		XiYan-SQL			OneSQL-v0.1-Qwen-7B			sqlcoder-7b-2			OmniSQL-7B		
Consulta	Descrição	EM	EX	VES	EM	EX	VES	EM	EX	VES	EM	EX	VES
1	Instituições em Campinas	1,00	1,00	0,9006	0,31	1,00	0,9039	0,31	0,00	0,0000	1,00	1,00	0,9776
2	Instituições em Minas Gerais	0,88	1,00	1,0586	0,28	0,00	0,0000	0,06	0,00	0,0000	0,59	0,00	0,0000
3	Instituições na região Nordeste	0,75	0,00	0,0000	0,27	0,00	0,0000	0,02	0,00	0,0000	1,00	1,00	0,9911
4	Distribuição de instituições por região	1,00	1,00	1,1081	0,38	0,00	0,0000	0,35	0,00	0,0000	0,40	0,00	0,0000
5	Média de instituições por região	1,00	1,00	1,0247	0,25	0,00	0,0000	0,75	0,00	0,0000	0,75	0,00	0,0000
6	Instituições públicas e privadas	0,83	0,50	1,0279	0,56	0,00	0,0000	0,31	1,00	1,2421	1,00	1,00	1,0568
7	Cursos da Faculdade de Rondônia	1,00	0,00	0,0000	1,00	1,00	0,8130	1,00	1,00	0,9679	1,00	1,00	0,8931
8	Engenharia Civil na Faculdade Sergipana	0,25	0,00	0,0000	0,19	0,00	0,0000	0,50	0,00	0,0000	0,56	0,00	0,0000
9	Cursos de bacharelado na Faculdade Cuiabá	1,00	1,00	0,9099	0,29	0,00	0,0000	0,46	1,00	0,8134	0,46	0,00	0,0000
10	Ciências Sociais na Faculdade Vitoriana	0,61	1,00	1,0508	0,27	0,00	0,0000	1,00	1,00	0,8118	1,00	1,00	0,9851
11	Cursos EAD na Faculdade Taboão	1,00	1,00	0,9916	0,42	1,00	0,3334	0,46	0,00	0,0000	0,42	0,00	0,0000
12	Média tempo integralização Faculdade Fidelis	1,00	1,00	1,0000	1,00	0,00	0,8132	0,31	0,00	0,0000	0,81	1,00	0,9327
13	Média tempo integralização Ciências Exatas	1,00	1,00	0,7866	0,50	0,00	0,0000	0,12	0,00	0,0000	0,50	0,00	0,0000
14	Docentes com doutorado na UERJ	1,00	1,00	0,9311	0,44	0,00	0,0000	0,25	0,00	0,0000	0,81	1,00	0,8791
15	Docentes com mestrado ou especialização na UNAMA	1,00	1,00	0,8796	0,56	0,00	0,0000	0,25	0,00	0,0000	0,56	0,00	0,0000
Média		**0,887**	**0,767**	**0,7113**	**0,447**	**0,200**	**0,1908**	**0,412**	**0,267**	**0,2550**	**0,723**	**0,467**	**0,4477**

No nível Médio, o *XiYan-SQL* apresentou o melhor desempenho, com médias de EM = 0,887, EX = 0,767 e VES = 0,7113, destacando-se em consultas como instituições em Campinas (EM = 1,00, EX = 1,00, VES = 0,9006) e distribuição de instituições por região (EM = 1,00, EX = 1,00, VES = 1,1081). O *OmniSQL-7B* teve desempenho razoável, com EM = 0,723, EX = 0,467 e VES = 0,4477, mas foi penalizado em consultas como cursos EAD na Faculdade Taboão (EX = 0,00). O *OneSQL-v0.1-Qwen-7B* e *sqlcoder-7b-2* tiveram os piores desempenhos, com médias de EM = 0,447 e 0,412, EX = 0,200 e 0,267, e VES = 0,1908 e 0,2550, respectivamente, devido a falhas frequentes em junções, como nas consultas sobre instituições na região Nordeste (EX = 0,00). A consulta sobre Engenharia Civil na Faculdade Sergipana foi desafiadora para todos os modelos, com EM baixo (0,25 a 0,56), refletindo dificuldades na identificação precisa do curso.

4.2.3 Nível Difícil

As consultas de alta complexidade, com múltiplas junções e agregações, desafiam os modelos em cenários exigentes. A Tabela 7 consolida os desempenhos, destacando a capacidade dos modelos no esquema INEP.

Tabela 7 – Resultados das Consultas de Nível Difícil

		XiYan-SQL			OneSQL-v0.1-Qwen-7B			sqlcoder-7b-2			OmniSQL-7B		
Consulta	Descrição	EM	EX	VES	EM	EX	VES	EM	EX	VES	EM	EX	VES
1	Instituições por organização acadêmica e região	1,00	1,00	1,1477	0,58	0,00	0,0000	0,43	0,00	0,0000	0,37	0,00	0,0000
2	Instituições por categoria administrativa e estado	0,63	0,00	0,0000	0,60	0,00	0,0000	0,38	0,00	0,0000	0,40	0,00	0,0000
3	Cursos por área e região	0,68	0,50	1,5435	0,38	0,00	0,0000	0,38	0,00	0,0000	0,44	0,00	0,0000
4	Docentes com doutorado por instituição e região	0,62	0,00	0,0000	0,02	0,00	0,0000	0,08	0,00	0,0000	0,15	0,00	0,0000
Média		**0,732**	**0,375**	**0,6728**	**0,395**	**0,000**	**0,0000**	**0,317**	**0,000**	**0,0000**	**0,340**	**0,000**	**0,0000**

No nível Difícil, o *XiYan-SQL* apresentou o melhor desempenho, com médias de EM = 0,732, EX = 0,375 e VES = 0,6728, destacando-se na consulta sobre instituições por organização acadêmica e região (EM = 1,00, EX = 1,00, VES = 1,1477). Os outros

modelos tiveram desempenhos significativamente inferiores, com *OmniSQL-7B* (EM = 0,340, EX = 0,000, VES = 0,0000), *OneSQL-v0.1-Qwen-7B* (EM = 0,395, EX = 0,000, VES = 0,0000), e *sqlcoder-7b-2* (EM = 0,317, EX = 0,000, VES = 0,0000) falhando em executar corretamente qualquer consulta (EX = 0,00). A consulta sobre instituições por categoria administrativa e estado foi particularmente desafiadora, com *XiYan-SQL* obtendo o maior EM (0,63), mas sem sucesso na execução (EX = 0,00). A eficiência do *XiYan-SQL* foi notável na consulta sobre cursos por área e região (VES = 1,5435), apesar de EX = 0,50.

4.3 Síntese Geral

Os quatro modelos apresentaram desempenhos variados, com o *XiYan-SQL* destacan do-se como o melhor modelo geral, com médias gerais de EM = 0,874, EX = 0,708 e VES = 0,708, conforme apresentado na Tabela 8. O *OmniSQL-7B* foi competitivo no nível Fácil (EM = 1,000, EX = 1,000, VES = 0,9635), mas caiu significativamente no nível Médio (EM = 0,723, EX = 0,467) e Difícil (EM = 0,340, EX = 0,000), resultando em médias gerais de EM = 0,709, EX = 0,479 e VES = 0,451. O *OneSQL-v0.1-Qwen-7B* e *sqlcoder-7b-2* tiveram desempenhos inferiores, com médias gerais de EM = 0,523 e 0,428, EX = 0,258 e 0,233, e VES = 0,237 e 0,228, respectivamente, devido a falhas consistentes em consultas complexas. O *XiYan-SQL* demonstrou maior consistência, com bom desempenho no nível Fácil (EM = 0,950, EX = 0,800), forte recuperação no nível Médio (EM = 0,887, EX = 0,767), e resiliência no nível Difícil (EM = 0,732, EX = 0,375), destacando-se pela eficiência em consultas como cursos por área e região (VES = 1,5435) e distribuição de instituições por região (VES = 1,1081).

5 CONCLUSÃO

Este trabalho avaliou o desempenho de quatro modelos de inteligência artificial para geração de consultas SQL no esquema INEP: *XiYan-SQL*, *OneSQL-v0.1-Qwen-7B*, *sqlcoder-7b-2* e *OmniSQL-7B*. A análise foi conduzida com base nas métricas *Exact Matching Accuracy* (EM), *Execution Accuracy* (EX) e *Valid Efficiency Score* (VES), aplicadas a consultas de diferentes níveis de dificuldade (Fácil, Médio e Difícil). Esta seção sumariza os principais resultados, discute suas implicações e sugere direções para desenvolvimentos futuros.

Os resultados demonstraram que o *XiYan-SQL* obteve o melhor desempenho geral, com médias gerais de EM = 0,874, EX = 0,708 e VES = 0,708. No nível Fácil, o modelo alcançou EM = 0,950 e EX = 0,800, mas foi superado pelo *OmniSQL-7B* (EM = 1,000, EX = 1,000). No nível Médio, o *XiYan-SQL* destacou-se com EM = 0,887 e EX = 0,767, superando todos os modelos em consultas como instituições em Campinas (EM = 1,00, EX = 1,00). No nível Difícil, o modelo manteve resiliência com EM = 0,732 e EX = 0,375, enquanto os outros modelos falharam em executar consultas corretamente (EX = 0,00). Comparado ao *OmniSQL-7B* (EM = 0,709, EX = 0,479, VES = 0,451), o *XiYan-SQL* foi superior em todos os níveis, exceto no Fácil, e apresentou VES consistentemente mais alto. O *OneSQL-v0.1-Qwen-7B* (EM = 0,523, EX = 0,258) e *sqlcoder-7b-2* (EM = 0,428, EX = 0,233) tiveram desempenhos limitados, especialmente em cenários complexos.

A superioridade do *XiYan-SQL* evidencia sua capacidade de compreender o esquema INEP e gerar consultas SQL precisas e eficientes, mesmo em contextos complexos.

Tabela 8 – Resumo das Métricas por Nível de Dificuldade

Nível/Modelo	EM	EX	VES
Fácil			
XiYan-SQL	0,950	0,800	0,7603
OneSQL-v0.1-Qwen-7B	0,950	0,800	0,7744
sqlcoder-7b-2	0,650	0,400	0,3939
OmniSQL-7B	**1,000**	**1,000**	**0,9635**
Médio			
XiYan-SQL	**0,887**	**0,767**	**0,7113**
OneSQL-v0.1-Qwen-7B	0,447	0,200	0,1908
sqlcoder-7b-2	0,412	0,267	0,2550
OmniSQL-7B	0,723	0,467	0,4477
Difícil			
XiYan-SQL	**0,732**	**0,375**	**0,6728**
OneSQL-v0.1-Qwen-7B	0,395	0,000	0,0000
sqlcoder-7b-2	0,317	0,000	0,0000
OmniSQL-7B	0,340	0,000	0,0000
Média Geral			
XiYan-SQL	**0,874**	**0,708**	**0,708**
OneSQL-v0.1-Qwen-7B	0,523	0,258	0,237
sqlcoder-7b-2	0,428	0,233	0,228
OmniSQL-7B	0,709	0,479	0,451

Sua eficiência computacional, refletida pelo VES médio de 0,708, sugere potencial para aplicações práticas em sistemas de consulta educacional. Contudo, limitações foram observadas, como erros em consultas como Engenharia Civil na Faculdade Sergipana (EM = 0,25, EX = 0,00) e instituições por categoria administrativa e estado (EX = 0,00), indicando desafios em condições ambíguas ou relações hierárquicas complexas.

Para trabalhos futuros, recomenda-se aproveitar o *XiYan-SQL* para desenvolver um sistema interativo de consulta educacional baseado em linguagem natural, como chatbots. Esse sistema poderia: (i) integrar interfaces conversacionais para interpretar perguntas em português; (ii) otimizar a geração de SQL para reduzir latência; e (iii) incorporar feedback do usuário para refinar consultas. Tais desenvolvimentos podem democratizar o acesso aos dados do Censo da Educação Superior, beneficiando gestores, pesquisadores e o público geral.

Este estudo contribui para o avanço de assistentes de banco de dados baseados em inteligência artificial, demonstrando que o *XiYan-SQL* é uma solução promissora para Text-to-SQL no contexto educacional brasileiro. Sua robustez e eficiência abrem caminho para sistemas inovadores de acesso a dados, consolidando o potencial dos modelos de linguagem em aplicações práticas.

5.1 Arquitetura Proposta para o Sistema de Consulta Educacional

Para atender à recomendação de trabalhos futuros, propõe-se uma arquitetura para um sistema interativo de consulta educacional baseado em linguagem natural, utilizando o XiYan-SQL como núcleo para geração de consultas SQL. Este sistema, estruturado como um chatbot, permitirá que usuários sem conhecimento em SQL acessem os dados do Censo da Educação Superior de forma intuitiva. A arquitetura, ilustrada na Figura 9, é composta por cinco componentes principais que trabalham em conjunto para processar perguntas, gerar consultas, acessar dados e refinar resultados.

- **Interface Conversacional**: Recebe perguntas em português via uma interface de chatbot (e.g., texto ou voz).
- **Módulo de Geração de SQL**: Emprega o XiYan-SQL para traduzir a entrada em linguagem natural em consultas SQL otimizadas, utilizando o esquema do banco de dados do Censo da Educação Superior.
- **Módulo de Acesso ao Banco de Dados**: Executa as consultas SQL geradas no banco de dados do Censo da Educação Superior, retornando os resultados em formato estruturado (e.g., JSON).
- **Módulo de Feedback do Usuário**: Permite que o usuário refine a consulta por meio de interações, como esclarecimentos ou reformulações, ajustando os parâmetros da consulta SQL para maior precisão.
- **Saída de Resultados**: Apresenta os dados ao usuário em formato compreensível, como tabelas, gráficos ou respostas textuais, adaptadas ao contexto da pergunta.

Esta arquitetura viabiliza a democratização do acesso aos dados do Censo da Educação Superior, permitindo que gestores, pesquisadores e o público geral obtenham informações educacionais de forma eficiente e intuitiva, consolidando o potencial do *XiYan-SQL* em aplicações práticas.

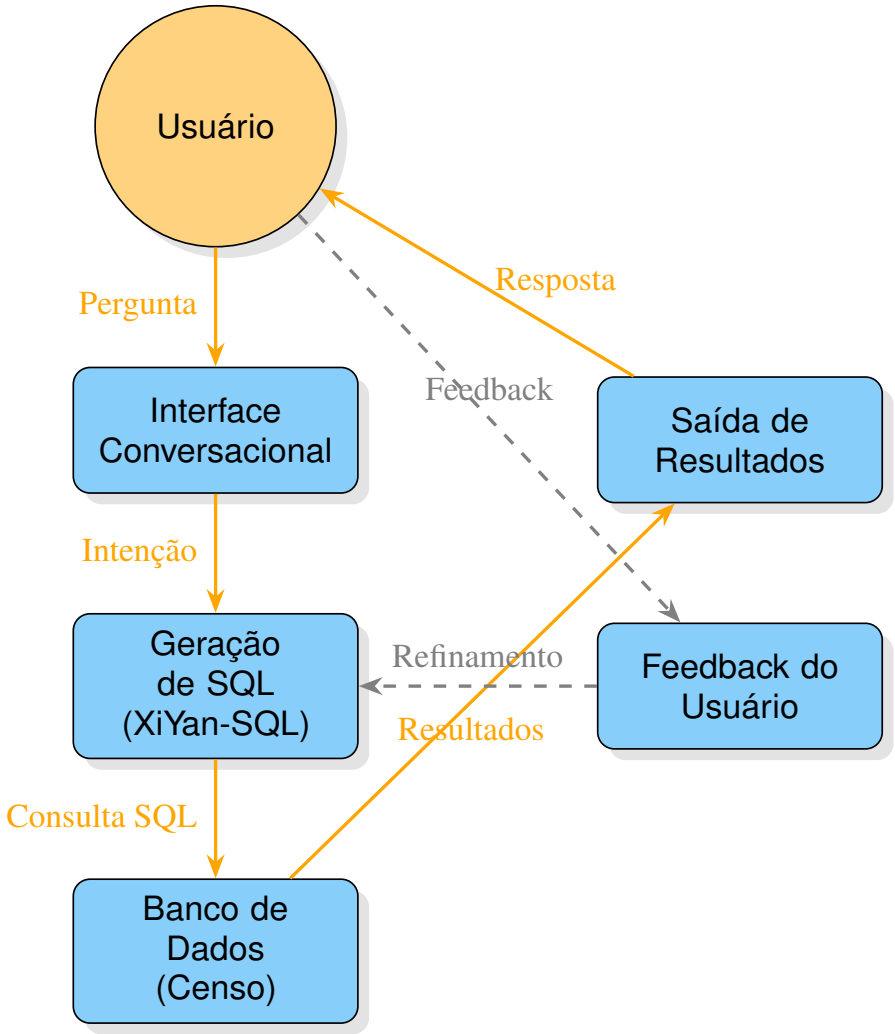

Figura 9 – Arquitetura do sistema de consulta educacional baseado em linguagem natural com XiYan-SQL.

REFERÊNCIAS

ANTONI, M. et al. Uso de linguagem natural para consulta de informações dos microdados do censo escolar brasileiro. Universidade Federal de Santa Maria, 2021. Citado 2 vezes nas páginas 11 e 24.

ASKARI, A.; POELITZ, C.; TANG, X. Magic: Generating self-correction guideline for in-context text-to-sql. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2025. v. 39, n. 22, p. 23433–23441. Citado na página 19.

BROWN, T. et al. Language models are few-shot learners. *Advances in neural information processing systems*, v. 33, p. 1877–1901, 2020. Citado na página 16.

CHOMSKY, N. *The minimalist program*. [S.l.]: MIT press, 2014. Citado na página 14.

CRUSE, A. Meaning in language: An introduction to semantics and pragmatics. 2004. Citado 2 vezes nas páginas 14 e 15.

DALE, R. Classical approaches to natural language processing. In: ____. *Handbook of Natural Language Processing, Second Edition*. United States: CRC Press, Taylor & Francis Group, 2010. (Chapman & Hall: CRC machine learning & pattern recognition series), p. 3–7. ISBN 9781420085921. Citado na página 13.

DENG, N.; CHEN, Y.; ZHANG, Y. Recent advances in text-to-sql: a survey of what we have and what we expect. *arXiv preprint arXiv:2208.10099*, 2022. Citado na página 24.

DEVLIN, J. et al. Bert: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. [S.l.: s.n.], 2019. p. 4171–4186. Citado 2 vezes nas páginas 15 e 18.

GAO, Y. et al. A preview of xiyan-sql: A multi-generator ensemble framework for text-to-sql. *arXiv preprint arXiv:2411.08599*, 2025. Citado na página 23.

GODDARD, C.; SCHALLEY, A. C. *Semantic Analysis*. 2010. Citado na página 14.

GUPTA, P. et al. Iqs-intelligent querying system using natural language processing. In: *2017 International conference of Electronics, Communication and Aerospace Technology (ICECA)*. [S.l.: s.n.], 2017. v. 2, p. 410–413. Citado na página 11.

HIPPISLEY, A. Lexical analysis. *Handbook of natural language processing*, v. 2, p. 31–58, 2010. Citado na página 14.

HOWARD, J.; RUDER, S. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018. Citado na página 16.

HU, E. J. et al. Lora: Low-rank adaptation of large language models. *ICLR*, v. 1, n. 2, p. 3, 2022. Citado na página 16.

JOSÉ, M. A.; COZMAN, F. G. mrat-sql+ gap: a portuguese text-to-sql transformer. In: SPRINGER. *Intelligent Systems: 10th Brazilian Conference, BRACIS 2021, Virtual Event, November 29–December 3, 2021, Proceedings, Part II 10*. [S.l.], 2021. p. 511–525. Citado 2 vezes nas páginas 11 e 24.

JURAFSKY, D.; MARTIN, J. H. Speech and language processing: An introduction to speech recognition, computational linguistics and natural language processing. *Upper Saddle River, NJ: Prentice Hall*, 2008. Citado na página 12.

KATSOGIANNIS-MEIMARAKIS, G.; KOUTRIKA, G. A survey on deep learning approaches for text-to-sql. *The VLDB Journal*, Springer, v. 32, n. 4, p. 905–936, 2023. Citado na página 23.

LI, J. et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, v. 36, p. 42330–42357, 2023. Citado 2 vezes nas páginas 19 e 23.

LIDDY, E. D. Natural language processing. 2001. Citado na página 13.

MANNING, C.; SCHUTZE, H. *Foundations of statistical natural language processing*. [S.l.]: MIT press, 1999. Citado na página 14.

MÜLLER, D. N. Processamento de linguagem natural. *Porto Alegre*, 2003. Citado na página 15.

POURREZA, M.; RAFIEI, D. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems*, v. 36, p. 36339–36348, 2023. Citado na página 24.

QIN, B. et al. A survey on text-to-sql parsing: Concepts, methods, and future directions. *arXiv preprint arXiv:2208.13629*, 2022. Citado 5 vezes nas páginas 16, 17, 18, 19 e 23.

SANDRA, D. *Morphological structure, lexical representation and lexical access*. [S.l.]: Taylor & Francis, 1994. v. 9. Citado na página 14.

SCHÜTZE, H.; MANNING, C. D.; RAGHAVAN, P. *Introduction to information retrieval*. [S.l.]: Cambridge University Press Cambridge, 2008. v. 39. Citado na página 14.

SHI, L. et al. A survey on employing large language models for text-to-sql tasks. *arXiv preprint arXiv:2407.15186*, 2024. Citado 4 vezes nas páginas 11, 15, 16 e 23.

SPROAT, R.; SHIH, C. A statistical method for finding word boundaries in chinese text. *Computer Processing of Chinese & Oriental Languages*, v. 4, n. 4, p. 336–351, 1990. Citado na página 13.

SUN, R. et al. Sql-palm: Improved large language model adaptation for text-to-sql (extended). *arXiv preprint arXiv:2306.00739*, 2023. Citado na página 24.

TAI, C.-Y. et al. Exploring chain-of-thought style prompting for text-to-sql. *arXiv preprint arXiv:2305.14215*, 2023. Citado na página 24.

TREVISAN, M. R. M. Avaliação de uma abordagem com large language models para consultas em linguagem natural a uma base pública de dados educacionais. Universidade Federal de Santa Maria, 2024. Citado na página 26.

VASWANI, A. et al. Attention is all you need. *Advances in neural information processing systems*, v. 30, 2017. Citado na página 15.

VASWANI, A. et al. *Attention Is All You Need*. 2023. Disponível em: <https://arxiv.org/abs/1706.03762>. Citado 2 vezes nas páginas 11 e 15.

WANG, B. et al. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *arXiv preprint arXiv:1911.04942*, 2019. Citado na página 18.

WEBSTER, J. J.; KIT, C. Tokenization as the initial phase in nlp. In: *COLING 1992 volume 4: The 14th international conference on computational linguistics*. [S.l.: s.n.], 1992. Citado na página 13.

WEI, J. et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022. Citado 2 vezes nas páginas 15 e 16.

WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, v. 35, p. 24824–24837, 2022. Citado na página 16.

YU, T. et al. Grappa: Grammar-augmented pre-training for table semantic parsing. *arXiv preprint arXiv:2009.13845*, 2020. Citado na página 18.

YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018. Citado 4 vezes nas páginas 16, 19, 23 e 27.

ZHU, X. et al. Large language model enhanced text-to-sql generation: A survey. *arXiv preprint arXiv:2410.06011*, 2024. Citado 4 vezes nas páginas 12, 22, 23 e 24.

APÊNDICE A – ESQUEMA DO BANCO DE DADOS

Para preservar a formatação e garantir a legibilidade, o código SQL para a criação das tabelas do banco de dados é apresentado abaixo.

```
CREATE TABLE INEP.REGIOES_IBGE (
    COD_REGIAO_IBGE INT PRIMARY KEY,
    DESCR_REGIAO_IBGE VARCHAR(255),
    SIGLA_REGIAO_IBGE VARCHAR(10)
);

CREATE TABLE INEP.UF_IBGE (
    UF_IBGE VARCHAR(20) PRIMARY KEY,
    NOME_UF_IBGE VARCHAR(255),
    COD_REGIAO_IBGE INT,
    LATITUDE DECIMAL(10,2),
    LONGITUDE DECIMAL(11,2),
    NOME_UF_IBGE_SEM_ACENTO VARCHAR(255),
    FOREIGN KEY (COD_REGIAO_IBGE) REFERENCES INEP.REGIOES_IBGE(
        COD_REGIAO_IBGE)
);

CREATE TABLE INEP.MESOREGIOES_IBGE (
    COD_MESOREGIAO_IBGE VARCHAR(20) PRIMARY KEY,
    NOME_MESOREGIAO_IBGE VARCHAR(255),
    COD_UF_IBGE VARCHAR(20),
    NUM_MESOREGIAO_IBGE INT,
    NOME_MESOREGIAO_IBGE_SEM_ACENTO VARCHAR(255),
    FOREIGN KEY (COD_UF_IBGE) REFERENCES INEP.UF_IBGE(UF_IBGE)
);

CREATE TABLE INEP.MICROREGIOES_IBGE (
    COD_MICROREGIAO_IBGE VARCHAR(20) PRIMARY KEY,
    NOME_MICROREGIAO_IBGE VARCHAR(255),
    COD_MESOREGIAO_IBGE VARCHAR(20),
    NUM_MICROREGIAO_IBGE INT,
    NOME_MICROREGIAO_IBGE_SEM_ACENTO VARCHAR(255),
    FOREIGN KEY (COD_MESOREGIAO_IBGE) REFERENCES INEP.
        MESOREGIOES_IBGE(COD_MESOREGIAO_IBGE)
);

CREATE TABLE INEP.MUNICIPIOS_IBGE (
    COD_IBGE VARCHAR(20) PRIMARY KEY,
    NOME_MUNICIPIO VARCHAR(255),
    COD_MICROREGIAO_IBGE VARCHAR(20),
    COD_MUNICIPIO VARCHAR(20),
    LATITUDE DECIMAL(10,2),
    LONGITUDE DECIMAL(11,2),
```

```
    POPULACAO INT,
    DENSIDADE DECIMAL(10,2),
    NOME_MUNICIPIO_SEM_ACENTO VARCHAR(255),
    FOREIGN KEY (COD_MICROREGIAO_IBGE) REFERENCES INEP.
        MICROREGIOES_IBGE(COD_MICROREGIAO_IBGE)
);

CREATE TABLE INEP.CENSO_CATEGORIAS_ADMINISTRATIVAS (
    ID_CATEGORIA_ADMINISTRATIVA INT PRIMARY KEY,
    DESCR_CATEGORIA_ADMINISTRATIVA VARCHAR(255)
);

CREATE TABLE INEP.CENSO_ORGANIZACOES_ACADEMICAS (
    ID_ORGANIZACAO_ACADEMICA INT PRIMARY KEY,
    DESCR_ORGANIZACAO_ACADEMICA VARCHAR(255)
);

CREATE TABLE INEP.CENSO_IES (
    COD_IES INT PRIMARY KEY,
    NOME_IES VARCHAR(255),
    SIGLA_IES VARCHAR(50),
    IN_CAPITAL INT CHECK (IN_CAPITAL IN (0, 1)),
    COD_MUNICIPIO VARCHAR(20),
    ID_CATEGORIA_ADMINISTRATIVA INT,
    ID_ORGANIZACAO_ACADEMICA INT,
    FOREIGN KEY (COD_MUNICIPIO) REFERENCES INEP.MUNICIPIOS_IBGE(
        COD_IBGE),
    FOREIGN KEY (ID_CATEGORIA_ADMINISTRATIVA) REFERENCES INEP.
        CENSO_CATEGORIAS_ADMINISTRATIVAS(
        ID_CATEGORIA_ADMINISTRATIVA),
    FOREIGN KEY (ID_ORGANIZACAO_ACADEMICA) REFERENCES INEP.
        CENSO_ORGANIZACOES_ACADEMICAS(ID_ORGANIZACAO_ACADEMICA)
);

CREATE TABLE INEP.CENSO_GRAUS_ACADEMICOS (
    ID_GRAU_ACADEMICO INT PRIMARY KEY,
    DESCR_GRAU_ACADEMICO VARCHAR(255)
);

CREATE TABLE INEP.CENSO_MODALIDADES_ENSINO (
    ID_MODALIDADE_ENSINO INT PRIMARY KEY,
    DESCR_MODALIDADE_ENSINO VARCHAR(255)
);

CREATE TABLE INEP.CENSO_NIVEL_ACADEMICO (
    ID_NIVEL_ACADEMICO INT PRIMARY KEY,
    DESCR_NIVEL_ACADEMICO VARCHAR(255)
);
```

```
CREATE TABLE INEP.CENSO_CINE_AREA_GERAL (
    COD_CINE_AREA_GERAL VARCHAR(20) PRIMARY KEY,
    DESCR_CINE_AREA_GERAL VARCHAR(255)
);

CREATE TABLE INEP.CENSO_CINE_AREA_ESPECIFICA (
    COD_CINE_AREA_ESPECIFICA VARCHAR(20) PRIMARY KEY,
    DESCR_CINE_AREA_ESPECIFICA VARCHAR(255),
    COD_CINE_AREA_GERAL VARCHAR(20),
    FOREIGN KEY (COD_CINE_AREA_GERAL) REFERENCES INEP.
        CENSO_CINE_AREA_GERAL(COD_CINE_AREA_GERAL)
);

CREATE TABLE INEP.CENSO_CINE_AREA_DETALHADA (
    COD_CINE_AREA_DETALHADA VARCHAR(20) PRIMARY KEY,
    DESCR_CINE_AREA_DETALHADA VARCHAR(255),
    COD_CINE_AREA_ESPECIFICA VARCHAR(20),
    FOREIGN KEY (COD_CINE_AREA_ESPECIFICA) REFERENCES INEP.
        CENSO_CINE_AREA_ESPECIFICA(COD_CINE_AREA_ESPECIFICA)
);

CREATE TABLE INEP.CENSO_CINE_ROTULO (
    COD_CINE_ROTULO VARCHAR(20) PRIMARY KEY,
    DESCR_CINE_ROTULO VARCHAR(255),
    COD_CINE_AREA_DETALHADA VARCHAR(20),
    AREA_TCU_1 VARCHAR(255),
    AREA_TCU_2 VARCHAR(255),
    FOREIGN KEY (COD_CINE_AREA_DETALHADA) REFERENCES INEP.
        CENSO_CINE_AREA_DETALHADA(COD_CINE_AREA_DETALHADA)
);

CREATE TABLE INEP.CENSO_CURSOS (
    COD_CURSO INT PRIMARY KEY,
    NOME_CURSO VARCHAR(255),
    COD_IES INT,
    ID_GRAU_ACADEMICO INT,
    ID_MODALIDADE_ENSINO INT,
    ID_NIVEL_ACADEMICO INT,
    ID_ATRIBUTO_INGRESSO INT,
    DT_INICIO_FUNCIONAMENTO DATE,
    COD_CINE_ROTULO VARCHAR(20),
    TERNEL_CURSO VARCHAR(50),
    CC_ANO INT,
    CC_SEME INT,
    COD_CINE_AREA_GERAL VARCHAR(20),
    TEMPO_INTEGRALIZACAO_ANOS INT,
    FOREIGN KEY (COD_IES) REFERENCES INEP.CENSO_IES(COD_IES),
    FOREIGN KEY (ID_GRAU_ACADEMICO) REFERENCES INEP.
        CENSO_GRAUS_ACADEMICOS(ID_GRAU_ACADEMICO),
```

```
    FOREIGN KEY (ID_MODALIDADE_ENSINO) REFERENCES INEP.
        CENSO_MODALidades_ENSINO(ID_MODALIDADE_ENSINO),
    FOREIGN KEY (ID_NIVEL_ACADEMICO) REFERENCES INEP.
        CENSO_NIVEL_ACADEMICO(ID_NIVEL_ACADEMICO),
    FOREIGN KEY (COD_CINE_ROTULO) REFERENCES INEP.
        CENSO_CINE_ROTULO(COD_CINE_ROTULO)
);

CREATE TABLE INEP.DADOS_IES_DOCENTES (
    NU_ANO_CENSO INT,
    CO_IES INT,
    QT_DOC_TOTAL INT,
    QT_DOC_EX_GRAD INT,
    QT_DOC_EX_SEM_GRAD INT,
    QT_DOC_EX_ESP INT,
    QT_DOC_EX_MEST INT,
    QT_DOC_EX_DOUT INT,
    PRIMARY KEY (NU_ANO_CENSO, CO_IES),
    FOREIGN KEY (CO_IES) REFERENCES INEP.CENSO_IES(COD_IES)
);

CREATE TABLE INEP.IND_FLUXO_IES (
    NU_ANO_CENSO INT,
    COD_CURSO INT,
    PERIODO VARCHAR(50),
    INGRESSO INT,
    NU_ANO_REFERENCIA INT,
    NU_PRAZO_INTEGRALIZACAO INT,
    NU_ANO_INTEGRALIZACAO INT,
    NU_PRAZO_ACOMPANHAMENTO INT,
    NU_ANO_MAXIMO_ACOMPANHAMENTO INT,
    QT_AGREGANTE INT,
    QT_PERMANENCIA INT,
    QT_CONCLUINTE INT,
    QT_DESISTENCIA INT,
    QT_FALECIDO INT,
    TAXA_PERMANENCIA DECIMAL(5, 2),
    TAXA_CONCLUSAO_ACUMULADA DECIMAL(5, 2),
    TAXA_DESISTENCIA_ACUMULADA DECIMAL(5, 2),
    TAXA_CONCLUSAO_ANUAL DECIMAL(5, 2),
    TAXA_DESISTENCIA_ANUAL DECIMAL(5, 2),
    PRIMARY KEY (NU_ANO_CENSO, COD_CURSO)
);
```

APÊNDICE B – PERGUNTAS E CONSULTAS SQL DE REFERÊNCIA

Este apêndice contém uma lista de perguntas em linguagem natural, classificadas por nível de dificuldade, e suas respectivas consultas SQL para extrair as respostas do banco de dados.

1 NÍVEL FÁCIL

Pergunta: Qual a sigla da instituição FACULDADE QUIRINÓPOLIS?

```
SELECT SIGLA_IES FROM INEP.CENSO_IES WHERE NOME_IES = 'FACULDADE
   QUIRINÓPOLIS';
```

Pergunta: Qual o nome da instituição de sigla FAREC?

```
SELECT NOME_IES FROM INEP.CENSO_IES WHERE SIGLA_IES = 'FAREC';
```

Pergunta: Quantas instituições estão localizadas em cidades capitais?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_IES WHERE IN_CAPITAL = 1;
```

Pergunta: A instituição Universidade Federal De Pernambuco está localizada em uma cidade capital?

```
SELECT IN_CAPITAL FROM INEP.CENSO_IES WHERE NOME_IES = 'UNIVERSIDADE
   FEDERAL DE PERNAMBUCO';
```

Pergunta: Quantos cursos possuem tempo de integralização de 4 anos?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_CURSOS WHERE
   TEMPO_INTEGRALIZACAO_ANOS = 4;
```

2 NÍVEL MÉDIO

Pergunta: Quantas instituições existem na cidade de Campinas?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_IES C JOIN INEP.
   MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.COD_IBGE WHERE M.
   NOME_MUNICIPIO = 'Campinas';
```

Pergunta: Quantas instituições existem no estado de Minas Gerais?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_IES C INNER JOIN INEP.
   MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.COD_IBGE JOIN INEP.
   MICROREGIOES_IBGE MI ON M.COD_MICROREGIAO_IBGE = MI.
   COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE ME ON MI.
   COD_MESOREGIAO_IBGE = ME.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON
    ME.COD_UF_IBGE = U.UF_IBGE WHERE U.NOME_UF_IBGE = 'Minas Gerais';
```

Pergunta: Quantas instituições existem na região Nordeste?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_IES C JOIN INEP.
   MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.COD_IBGE JOIN INEP.
   MICROREGIOES_IBGE MI ON M.COD_MICROREGIAO_IBGE = MI.
   COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE ME ON MI.
   COD_MESOREGIAO_IBGE = ME.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON
    ME.COD_UF_IBGE = U.UF_IBGE JOIN INEP.REGIOES_IBGE R ON U.
   COD_REGIAO_IBGE = R.COD_REGIAO_IBGE WHERE R.DESCR_REGIAO_IBGE = '
   Nordeste';
```

Pergunta: Qual a distribuição de instituições por região?

```
SELECT R.DESCR_REGIAO_IBGE, COUNT(*) AS total FROM INEP.CENSO_IES C
   JOIN INEP.MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.COD_IBGE JOIN
   INEP.MICROREGIOES_IBGE MI ON M.COD_MICROREGIAO_IBGE = MI.
   COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE ME ON MI.
   COD_MESOREGIAO_IBGE = ME.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON
    ME.COD_UF_IBGE = U.UF_IBGE JOIN INEP.REGIOES_IBGE R ON U.
   COD_REGIAO_IBGE = R.COD_REGIAO_IBGE GROUP BY R.DESCR_REGIAO_IBGE;
```

Pergunta: Qual a média de instituições entre as regiões?

```
SELECT AVG(total) AS media FROM (SELECT R.DESCR_REGIAO_IBGE, COUNT(*)
   AS total FROM INEP.CENSO_IES C JOIN INEP.MUNICIPIOS_IBGE M ON C.
   COD_MUNICIPIO = M.COD_IBGE JOIN INEP.MICROREGIOES_IBGE MI ON M.
   COD_MICROREGIAO_IBGE = MI.COD_MICROREGIAO_IBGE JOIN INEP.
   MESOREGIOES_IBGE ME ON MI.COD_MESOREGIAO_IBGE = ME.
   COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON ME.COD_UF_IBGE = U.
   UF_IBGE JOIN INEP.REGIOES_IBGE R ON U.COD_REGIAO_IBGE = R.
   COD_REGIAO_IBGE GROUP BY R.DESCR_REGIAO_IBGE) AS subquery;
```

Pergunta: Quantas instituições de nível superior são de administração pública e quantas são privadas?

```
SELECT SUM(CASE WHEN cca.ID_CATEGORIA_ADMINISTRATIVA = 1 THEN 1 ELSE 0
    END) AS publicas, SUM(CASE WHEN cca.ID_CATEGORIA_ADMINISTRATIVA =
   2 THEN 1 ELSE 0 END) AS privadas FROM INEP.CENSO_IES ies JOIN INEP.
   CENSO_CATEGORIAS_ADMINISTRATIVAS cca ON ies.
   ID_CATEGORIA_ADMINISTRATIVA = cca.ID_CATEGORIA_ADMINISTRATIVA;
```

Pergunta: Quantos cursos são ofertados pela instituição Faculdade De Rondônia?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_CURSOS CU JOIN INEP.CENSO_IES
    C ON CU.COD_IES = C.COD_IES WHERE C.NOME_IES = 'FACULDADE DE RONDÔ
   NIA';
```

Pergunta: O curso Engenharia Civil é ofertado pela instituição Faculdade Sergipana?

```
SELECT CASE WHEN COUNT(*) > 0 THEN 'Sim' ELSE 'Não' END AS ofertado
   FROM INEP.CENSO_CURSOS CU JOIN INEP.CENSO_IES C ON CU.COD_IES = C.
   COD_IES WHERE C.NOME_IES = 'FACULDADE SERGIPANA' AND CU.NOME_CURSO
   = 'Engenharia Civil';
```

Pergunta: Quantos cursos de grau bacharelado são ofertados pela instituição Faculdade Cuiabá?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_CURSOS CU JOIN INEP.CENSO_IES
    C ON CU.COD_IES = C.COD_IES JOIN INEP.CENSO_GRAUS_ACADEMICOS G ON
   CU.ID_GRAU_ACADEMICO = G.ID_GRAU_ACADEMICO WHERE C.NOME_IES = '
   FACULDADE CUIABÁ' AND G.DESCR_GRAU_ACADEMICO = 'Bacharelado';
```

Pergunta: Quantos cursos da área do conhecimento Ciências Sociais Aplicadas são ofertados pela instituição Faculdade Vitoriana De Tecnologia?

```
SELECT COUNT(*) AS total FROM INEP.CENSO_CURSOS CU JOIN INEP.CENSO_IES
    C ON CU.COD_IES = C.COD_IES JOIN INEP.CENSO_CINE_ROTULO CR ON CU.
   COD_CINE_ROTULO = CR.COD_CINE_ROTULO JOIN INEP.
   CENSO_CINE_AREA_DETALHADA CD ON CR.COD_CINE_AREA_DETALHADA = CD.
   COD_CINE_AREA_DETALHADA JOIN INEP.CENSO_CINE_AREA_ESPECIFICA CE ON
   CD.COD_CINE_AREA_ESPECIFICA = CE.COD_CINE_AREA_ESPECIFICA JOIN INEP
```

```
.CENSO_CINE_AREA_GERAL CG ON CE.COD_CINE_AREA_GERAL = CG.
COD_CINE_AREA_GERAL WHERE C.NOME_IES = 'FACULDADE VITORIANA DE
TECNOLOGIA' AND CG.DESCR_CINE_AREA_GERAL = 'Ciências Sociais
Aplicadas';
```

Pergunta: Quantos cursos são ofertados pela modalidade Curso a distância pela instituição Faculdade Taboão?

```
SELECT COUNT(*) AS total_cursos_ead FROM INEP.CENSO_CURSOS CUR JOIN
INEP.CENSO_IES IES ON CUR.COD_IES = IES.COD_IES JOIN INEP.
CENSO_MODALIDADES_ENSINO MOD ON CUR.ID_MODALIDADE_ENSINO = MOD.
ID_MODALIDADE_ENSINO WHERE IES.NOME_IES = 'FACULDADE TABOÃO' AND
MOD.DESCR_MODALIDADE_ENSINO = 'Curso a distância';
```

Pergunta: Qual a média do tempo de integralização dos cursos da instituição Faculdade Fidelis?

```
SELECT AVG(TEMPO_INTEGRALIZACAO_ANOS) AS media FROM INEP.CENSO_CURSOS
CU JOIN INEP.CENSO_IES C ON CU.COD_IES = C.COD_IES WHERE C.NOME_IES
 = 'FACULDADE FIDELIS';
```

Pergunta: Qual a média do tempo de integralização dos cursos da área do conhecimento Ciências Exatas e da Terra?

```
SELECT AVG(TEMPO_INTEGRALIZACAO_ANOS) AS media_tempo_integralizacao
FROM INEP.CENSO_CURSOS WHERE COD_CINE_AREA_GERAL = '04';
```

Pergunta: Quantos docentes na instituição Universidade Do Estado Do Rio De Janeiro têm título de doutorado?

```
SELECT SUM(QT_DOC_EX_DOUT) AS total_doutores FROM INEP.
DADOS_IES_DOCENTES D JOIN INEP.CENSO_IES C ON D.CO_IES = C.COD_IES
WHERE C.NOME_IES = 'UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO';
```

Pergunta: Quantos docentes na instituição Universidade Da Amazônia têm título de mestrado ou especialização?

```
SELECT SUM(QT_DOC_EX_MEST + QT_DOC_EX_ESP) AS total FROM INEP.
DADOS_IES_DOCENTES D JOIN INEP.CENSO_IES C ON D.CO_IES = C.COD_IES
WHERE C.NOME_IES = 'UNIVERSIDADE DA AMAZÔNIA';
```

3 NÍVEL DIFÍCIL

Pergunta: Qual a distribuição de instituições, por tipo de organização acadêmica, por região IBGE?

```
SELECT R.DESCR_REGIAO_IBGE, OA.DESCR_ORGANIZACAO_ACADEMICA, COUNT(*)
AS total FROM INEP.CENSO_IES C JOIN INEP.
CENSO_ORGANIZACOES_ACADEMICAS OA ON C.ID_ORGANIZACAO_ACADEMICA = OA
.ID_ORGANIZACAO_ACADEMICA JOIN INEP.MUNICIPIOS_IBGE M ON C.
COD_MUNICIPIO = M.COD_IBGE JOIN INEP.MICROREGIOES_IBGE MI ON M.
COD_MICROREGIAO_IBGE = MI.COD_MICROREGIAO_IBGE JOIN INEP.
MESOREGIOES_IBGE ME ON MI.COD_MESOREGIAO_IBGE = ME.
COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON ME.COD_UF_IBGE = U.
UF_IBGE JOIN INEP.REGIOES_IBGE R ON U.COD_REGIAO_IBGE = R.
COD_REGIAO_IBGE GROUP BY R.DESCR_REGIAO_IBGE, OA.
DESCR_ORGANIZACAO_ACADEMICA;
```

Pergunta: Qual a distribuição de instituições, por tipo de administração, por estado?

```
SELECT U.NOME_UF_IBGE, CA.DESCR_CATEGORIA_ADMINISTRATIVA, COUNT(*) AS
    total FROM INEP.CENSO_IES C JOIN INEP.
    CENSO_CATEGORIAS_ADMINISTRATIVAS CA ON C.
    ID_CATEGORIA_ADMINISTRATIVA = CA.ID_CATEGORIA_ADMINISTRATIVA JOIN
    INEP.MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.COD_IBGE JOIN INEP.
    MICROREGIOES_IBGE MI ON M.COD_MICROREGIAO_IBGE = MI.
    COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE ME ON MI.
    COD_MESOREGIAO_IBGE = ME.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON
     ME.COD_UF_IBGE = U.UF_IBGE GROUP BY U.NOME_UF_IBGE, CA.
    DESCR_CATEGORIA_ADMINISTRATIVA;
```

Pergunta: Qual a distribuição de cursos, por área do conhecimento, por região IBGE?

```
SELECT R.DESCR_REGIAO_IBGE, CG.DESCR_CINE_AREA_GERAL, COUNT(*) AS
    total FROM INEP.CENSO_CURSOS CU JOIN INEP.CENSO_IES C ON CU.COD_IES
     = C.COD_IES JOIN INEP.MUNICIPIOS_IBGE M ON C.COD_MUNICIPIO = M.
    COD_IBGE JOIN INEP.MICROREGIOES_IBGE MI ON M.COD_MICROREGIAO_IBGE =
     MI.COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE ME ON MI.
    COD_MESOREGIAO_IBGE = ME.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE U ON
     ME.COD_UF_IBGE = U.UF_IBGE JOIN INEP.REGIOES_IBGE R ON U.
    COD_REGIAO_IBGE = R.COD_REGIAO_IBGE JOIN INEP.CENSO_CINE_ROTULO CR
    ON CU.COD_CINE_ROTULO = CR.COD_CINE_ROTULO JOIN INEP.
    CENSO_CINE_AREA_DETALHADA CD ON CR.COD_CINE_AREA_DETALHADA = CD.
    COD_CINE_AREA_DETALHADA JOIN INEP.CENSO_CINE_AREA_ESPECIFICA CE ON
    CD.COD_CINE_AREA_ESPECIFICA = CE.COD_CINE_AREA_ESPECIFICA JOIN INEP
    .CENSO_CINE_AREA_GERAL CG ON CE.COD_CINE_AREA_GERAL = CG.
    COD_CINE_AREA_GERAL GROUP BY R.DESCR_REGIAO_IBGE, CG.
    DESCR_CINE_AREA_GERAL;
```

Pergunta: Qual o percentual de docentes com doutorado, por instituição, por região IBGE?

```
SELECT cies.NOME_IES AS Instituicao, reg.DESCR_REGIAO_IBGE AS Regiao,
    (SUM(dados.QT_DOC_EX_DOUT) * 100.0 / SUM(dados.QT_DOC_TOTAL)) AS
    Percentual_Doutorado FROM INEP.DADOS_IES_DOCENTES dados JOIN INEP.
    CENSO_IES cies ON dados.CO_IES = cies.COD_IES JOIN INEP.
    MUNICIPIOS_IBGE mun ON cies.COD_MUNICIPIO = mun.COD_IBGE JOIN INEP.
    MICROREGIOES_IBGE micr ON mun.COD_MICROREGIAO_IBGE = micr.
    COD_MICROREGIAO_IBGE JOIN INEP.MESOREGIOES_IBGE meso ON micr.
    COD_MESOREGIAO_IBGE = meso.COD_MESOREGIAO_IBGE JOIN INEP.UF_IBGE uf
     ON meso.COD_UF_IBGE = uf.UF_IBGE JOIN INEP.REGIOES_IBGE reg ON uf.
    COD_REGIAO_IBGE = reg.COD_REGIAO_IBGE WHERE dados.QT_DOC_TOTAL > 0
    GROUP BY cies.NOME_IES, reg.DESCR_REGIAO_IBGE ORDER BY reg.
    DESCR_REGIAO_IBGE, cies.NOME_IES;
```